Robust optimal control of a lunar lander using automatic scenario generation and local reduction

Author

Y. Molano Gomez

CID: 01714721

Supervised by

Prof. E. Kerrigan

Second Marker

Dr Davide Amato

Industry Supervisor

Mr Alexander Popov

A Thesis submitted in fulfillment of requirements for the degree of
Master of Engineering in Aeronautical Engineering with a Year in Industry

Department of Aeronautics
Imperial College London

2024

Abstract

This master's thesis investigates the application of the local reduction robust optimal control method for lunar landers. It outlines the challenges faced in achieving pinpoint landing and underscores the necessity given current aspirations. The development of robust optimal control methodologies is proposed to address these challenges, specifically focusing on a novel local reduction approach. The study aims to demonstrate its applicability to soft landing scenarios, assess its accuracy and computational efficiency compared to state-of-the-art methods, and evaluate its ability to handle all types of uncertainties in lunar landing.

To achieve these goals, the thesis details the development of a nonlinear, non-convex trajectory optimization model, including its validation and efforts to improve convergence. It identifies and characterizes the main uncertainties in lunar landing, discussing their nature and the best strategies to model them. The development of the novel local reduction algorithm is then outlined, examining its properties such as convergence, scenario handling, and execution speed improvements. Additionally, methods to propagate the trajectory in open-loop and closed-loop are developed and analyzed using Monte Carlo simulations.

Results demonstrate that the proposed method significantly enhances landing accuracy, reducing the landing ellipse size by more than one order of magnitude in open-loop scenarios compared to the nominal case. It effectively mitigates uncertainties, reducing the standard deviation by two orders of magnitude, and accurately predicts the minimum fuel mass required in the worst case. Crucially, it provides highly accurate predictions for disturbances found in Monte Carlo simulations for each control input in near real-time, enabling numerous possibilities for real-time validation. For closed-loop approach, the method indicates that, with improved execution speed to run in decreasing horizon, pinpoint landing accuracy within centimeters is achievable.

Finally, the thesis explores new research avenues and potential applications of the local reduction method beyond soft landing. These include neural network training, accurate real-time validation and verification, and robust, adaptable trajectory planning in unforeseen scenarios.

Declaration of Originality

I hereby declare that the work presented in this thesis is my own unless otherwise stated. To the best of my knowledge the work is original and ideas developed in collaboration with others have been appropriately referenced.

Copyright Declaration

The copyright of this thesis rests with the author and is made available under a Creative Commons Attribution Non-Commercial No Derivatives licence. Researchers are free to copy, distribute or transmit the thesis on the condition that they attribute it, that they do not use it for commercial purposes and that they do not alter, transform or build upon it. For any reuse or redistribution, researchers must make clear to others the licence terms of this work.

Acknowledgments

I would like to start by expressing my gratitude to Professor Kerrigan for agreeing to supervise this project in collaboration with Airbus Defence & Space. While I have always found control appealing, it was this project that immersed me in the world of trajectory optimization and optimal control, ultimately igniting my passion for robotics and my desire to pursue a PhD. This experience has been an incredible journey of growth and learning, one I am sure I will always look back on fondly. Secondly, thank you to Dr. Amato for his interest in this project's outcome and his valuable input.

I also want to thank Mr. Arribas and Mr. Popov for accepting my request to collaborate with Airbus Defence & Space on this project, requesting a very unusual frequent visitor pass on my behalf and making me feel welcome at the Airbus offices.

Finally, I would like to thank my family and all the wonderful friends I have made during my time at Imperial College. They have made my experience at Imperial more enjoyable than challenging, although challenging enough. I am also grateful to the broader Imperial College community. Over the past five years, this wonderful institution has significantly shaped who I am, and I will always look back at my time here as some of the best times of my life.

Contents

Abstract **i**

Declaration of Originality **iii**

Copyright Declaration **v**

Acknowledgments **vi**

List of Figures **ix**

List of Tables **x**

1 Introduction & Motivation **1**
1.1 Current challenges with lunar landing 1
1.2 The promise of robust methods 1

2 Literature review & Theoretical Background **2**
2.1 Advancements in Autonomous Guidance Navigation (AGN) 2
2.2 Robust optimal control 2
2.2.1 Optimal control theory 3
2.2.2 Collocation techniques 3
2.2.3 Convexification 4
2.2.4 Robust optimal control: local reduction method 4

3 Outline of the project **5**
3.1 Research question 5
3.2 Convex vs non-convex model 5
3.3 Using Julia + JuMP 5
3.4 Key objectives 6
3.4.1 Development of a non-linear non-convex model 6
3.4.2 Identification and characterisation of key sources of uncertainty in lunar landing 6
3.4.3 Development of local reduction algorithm to find robust trajectories 6
3.4.4 Benchmark development to compare robust model to baseline 7

4 Development of Nonlinear, Non-Convex Model **8**
4.1 ESA Model 8
4.1.1 Key Characteristics of Argonaut 8
4.1.2 Optimal Control Problem Formulation 9
4.1.3 Verification 11
4.1.4 Limitations of the model 13
4.2 Quaternion Formulation 13

4.2.1 Differences between the models . . . 13
4.2.2 Optimal control formulation . . . 14
4.2.3 Results & Verification . . . 15
4.2.4 Hazards to avoid . . . 17

5 Uncertainty characterisation **19**
5.1 Key sources of uncertainty . . . 19
5.2 Compatibility with local reduction . . . 19
5.2.1 Distributions . . . 20
5.2.2 Uncertainty as a function of altitude/time . . . 20
5.3 Chosen uncertainties to include in the model . . . 21

6 Local reduction development **22**
6.1 The local reduction algorithm . . . 22
6.1.1 Changes to minimisation model . . . 23
6.1.2 Maximisation models . . . 23
6.1.3 Free-final time vs. fixed-final time . . . 24
6.1.4 JuMP Implementation . . . 26
6.2 Behaviour of algorithm . . . 26
6.2.1 Adding the worst scenario vs all the scenarios above a threshold . . . 26
6.2.2 Number of scenarios vs overall perturbations . . . 27
6.3 Refinements to the model . . . 27
6.3.1 Tuning parameters . . . 27
6.3.2 Limiting number of scenarios . . . 28

7 Results & Discussion **31**
7.1 Open-Loop Response - Key Findings . . . 31
7.1.1 Accuracy in predictions . . . 32
7.1.2 Scenarios considered . . . 33
7.1.3 Sensitivity analysis . . . 34
7.2 Closed-Loop Response . . . 34
7.2.1 Feedback vs computational time . . . 35
7.2.2 Alternative formulations . . . 37
7.2.3 Propagation . . . 37

8 Future Work & Use Cases **38**

Conclusions **40**

Bibliography **41**

List of Figures

4.1 Descent & Landing Scenario Argonaut 8
4.2 Trajectory from original model shown. 12
4.3 Discretization error i) absolute error for altitude, ii) relative error per timestep for all variables 12
4.4 Initial trajectory observed for Problem 2 16
4.5 Different initialisations for the final time, all converging towards 4.29s 17

5.1 Altitude error before and after switch-off of NDL. This figure is taken from [22] . . 20

6.1 Deviation from final conditions with number of scenarios. 26
6.2 Deviation from final conditions with number of scenarios. 27
6.3 Perturbation in quaternion, where the objective is to be colinear with the x-axis direction. 29

7.1 Comparison between the nominal control input and the local reduction input, for a 2000 simulation Monte Carlo. 32
7.2 $w_{\mathbf{T}_\mathcal{B}}$ for all of the scenarios considered 33
7.3 Sensivity analysis on the Open-Loop Local Reduction control input 34
7.4 Comparison between the nominal control input and the local reduction input, for a 2000 simulation Monte Carlo in closed-loop. 36

List of Tables

4.1 Descent and landing parameters and constraints [20]. 9
4.2 Average discretisation error per timestep for each variable 13
4.3 Conditions used for second model 16
4.4 Average discretization error per timestep for each variable 16

5.1 Table showing the sensors used for attitude estimation with the usual level of accuracy that can be expected. [22] 19
5.2 Uncertainties to include in the model 21

6.1 Maximisation model objective values 25
6.2 Predicted deviations from local reduction algorithm after the addition of 50 scenarios. 28
6.3 Relaxed constraints for optimisation problem 29

7.1 Results from 2000 Monte-carlo runs, comparing the final conditions achieved. . . 31
7.2 Predicted values by the local reduction in Chapter 6 with the real values from the Monte Carlo simulation. 33
7.3 Results from 2000 Monte Carlo runs, comparing the final conditions achieved for the closed loop 35

1 Introduction & Motivation

1.1 Current challenges with lunar landing

In Entry, Descent, and Landing (EDL), precision is measured by a *landing ellipse*: the region where it is 99% likely that the vehicle will land. Achieving precision landing remains elusive; of the latest 5 lunar missions to attempt landing, only the Chandrayaan-3 mission has done so successfully [1]. Even so, this mission has a landing ellipse of 4 km by 2.4 km [2], and achieved a landing $400m$ from the intended target. To make the current aspirations for long-term settlements in the Moon achievable, many landers will have to land in very close proximity accurately and safely. Therefore, achieving pin-point landing and reducing landing ellipses is imperative [3].

Why is this landing ellipse so expansive? One reason are the uncertainties introduced by variation in the performance of the propulsion system, sensor inaccuracies as well as uncertainties in the landing position due to surface conditions. Limitations in the guidance and control algorithms, particularly in adapting to real-time changes and uncertainties, can result in deviations from the target.

1.2 The promise of robust methods

Robust optimal control (ROC) aims to find control inputs that minimize deviations and optimize performance despite the presence of bounded uncertainties, thereby enhancing the potential for precise and reliable landing.

Developing robust methods also has the potential of improving the payload mass of the missions; in space missions, fuel that must be carried is not only defined by the amount needed, but by how much uncertainty there is in this value. ROC methodologies can decrease the variability in fuel used due to uncertainty and also improve estimates for it, potentially leading to a large improvement in payload fraction mass.

Finally, robust methods can decrease the rate of failures; very recently, an unexpected decrease in thruster power cause Japan's SLIM mission to land upside down [4]. Assuming this variation was bounded and modelled, a robust method could find a control input that does not rely as heavily on a strong attitude correcting burn. Overall, the use of robust methods could make pinpoint landing a reality, increase payload fraction mass, and decrease mission failure rates. This project aims to advance the development of these methods.

2 Literature review & Theoretical Background

2.1 Advancements in Autonomous Guidance Navigation (AGN)

In the Apollo missions, polynomial guidance was used to calculate acceleration based on the remaining flight time [5]. However, this method couldn't handle complex constraints [6]. The Mars Science Laboratory (MSL) advanced this approach by using three-dimensional polynomial trajectories computed online, significantly improving precision [7].

Direct methods were employed to solve three-degree-of-freedom (3-DoF) nonlinear programming (NLP) problems with a fuel-optimality objective, introducing more complex constraints. While these methods exhibited high accuracy, they faced challenges in real-time performance and convergence [8] [9]. To improve upon this, convex optimization was utilized, transforming the problem into semi-definite programming (SDP) and second-order cone programming (SOCP). It was demonstrated that lossless convexification provided the same solution as the original problem based on the Pontryagin maximum principle [10].

Successive Convex Programming (SCP) addressed more complex scenarios, ensuring continuous-time convergence and optimality by combining trust region methods and virtual control variables. However, SCP only ensured equivalence to the original problem when minimum fuel was taken as the objective and the modulus of the thrust vector equaled the relaxation variable. Similarly, simulations have shown that reasonable initial value guesses can help find a local solution for an NLP with similar computing speed and convergence to convex optimization methods.

Receding Horizon Control (RHC) and Model Predictive Control (MPC) generate real-time control commands, ensuring methodologies that guarantee real-time performance. For other guidance methods, the zero-effort-miss/zero-effort-velocity (ZEM/ZEV) feedback guidance law has been employed to compute acceleration control commands for tracking discrete waypoints and avoiding obstacles. This method also generates feedback guidance commands with terminal attitude constraints by introducing auxiliary control variables [11]. However, ZEM/ZEV can only derive guidance commands when omitting aerodynamic forces.

The state-of-the-art for pinpoint landing is therefore currently lossless convex formulations in receding horizon control. These techniques have been successfully used for pinpoint landing on Earth by SpaceX [3].

2.2 Robust optimal control

Robust optimal control is a framework designed to ensure that the control system can handle a range of uncertainties and disturbances, maintaining desired performance and stability. Traditional approaches have included min-max optimisation, scenario-based optimal control, and stochastic optimal control.

Min-Max MPC considers a maximisation optimisation to find the worst-case scenario, and a minimisation step to minimise this cost. As it considers nested optimisations, it has a high dimen-

sionality and complexity, and is generally considered too computationally expensive for applicability [12]. Stochastic MPC handles uncertainties as random variables and aims to minimize their expected value, offering a balance between performance and robustness, yet it still requires significant computational resources and relies on accurate probabilistic models of uncertainty. Scenario-based MPC considers a finite sample of scenarios, offering potential computational benefits by reducing the problem size, but its robustness depends heavily on the representativeness of the chosen scenarios, which can be challenging to ensure [13]. Common scenarios to consider include extreme scenarios or random scenarios.

Tube-MPC provides a compelling alternative, using a tube around the predicted trajectory to ensure constraint satisfaction, thereby balancing robustness and computational efficiency. This method offers robust stability and performance while mitigating the computational burden compared to min-max MPC, though it still requires careful tuning of the tube parameters to maintain performance [14]. An alternative approach is the application of local reduction [15], effectively combining aspects of scenario-based and min-max MPC. By iteratively finding the uncertainty variables that most violate the constraints, additional constraints can be incorporated to account for worst-case scenarios, reducing the problem from infinite to finite optimization and mitigating the computational complexity while maintaining robustness. It has been proven that it can find a smaller number of scenarios that lead to more robust results than typical scenario-based approach like the extreme scenarios approach. This project will investigate whether local reduction can be applied to the soft landing optimal problem by using lunar landing as a case study.

2.2.1 Optimal control theory

This section will very briefly describe optimal control theory. It is based on very good tutorial papers, which can be consulted for more depth in the subject.[16] [17].

Optimal control theory is the basis of all trajectory optimisation problems, where the goal is to find an optimal input trajectory for the problem:

$$\begin{aligned}
&\min_{t_f,\boldsymbol{u}} L_f\left(\boldsymbol{x}\left(t_f\right), t_f\right) + \int_0^{t_f} L(\boldsymbol{x}(\tau), \boldsymbol{u}(\tau), \tau)\mathrm{d}\tau \text{ s.t.} \\
&\dot{\boldsymbol{x}}(t) = \boldsymbol{f}(\boldsymbol{x}(t), \boldsymbol{u}(t), t), \forall t \in [0, t_f], \\
&\boldsymbol{g}(\boldsymbol{x}(t), \boldsymbol{u}(t), t) \leq 0, \forall t \in [0, t_f], \\
&\boldsymbol{b}\left(\boldsymbol{x}(0), \boldsymbol{x}\left(t_f\right), t_f\right) = 0.
\end{aligned} \tag{2.1}$$

where $\boldsymbol{x} : [0, t_f] \to \mathbb{R}^{n_x}$ is the state trajectory, $u : [0, t_f] \to \mathbb{R}^{n_u}$ is the input trajectory and $t_f \in$ is the final time. The state evolves according to the dynamics $\boldsymbol{f} : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \times \mathbb{R} \to \mathbb{R}^{n_x}$, and satisfies a set of constraints $\boldsymbol{g} : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \times \mathbb{R} \to \mathbb{R}^{n_c}$. At the boundaries, the state satisfies conditions $\boldsymbol{b} : \mathbb{R}^{n_x} \times \mathbb{R}^{n_x} \times \mathbb{R} \to \mathbb{R}^{n_b}$. The quality of the trajectory is measured by a cost function that may combine a running cost $L : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \times \mathbb{R} \to \mathbb{R}$ and/or a terminal cost $L_f : \mathbb{R}^{n_x} \times \mathbb{R} \to \mathbb{R}$.

When compared to a typical parameter optimisation problem, the constraints encode a physical process governed by ordinary differential equations (ODEs). Secondly, as time is continous, the input trajectory has an infinite number of design parameters, making the problem intractable. To solve this, there are two approaches: direct and indirect methods.

2.2.2 Collocation techniques

Indirect methods attempt to solve the problem by solving the necessary conditions of optimality, whilst direct methods numerically solve the problem as a parameter optimisation. This section will focus on direct methods, as it is the most common used for a powered descent problem and the one used in this project.

In direct methods, the problem is discretised, reducing the semi-infinite problem into a finite dimensional problem. This is achieved by dividing time into a grid of N nodes, at which the constraints are satisfied, whilst the values in between are typically interpolated by splines. Some common methods include trapezoidal collocation and Hermite-Simpson collocation.

2.2.3 Convexification

At this point, the problem generally becomes a nonlinear program (NLP). This class of problems can be solved by algorithms most often comprising of a primal-dual interior point method (IPM). Whilst there are good algorithms for non-convex problems, formulating the problem in a convex form has many advantages. If a function is convex, global statements can be made from local function evaluations, guaranteeing that a global optimum can be found and the maximum iteration count identified. This is very beneficial for online use-cases, and for many problems, convexification can happen losslessly; the solution set is neither reduced nor expanded. However, this is not true for all cases.

2.2.4 Robust optimal control: local reduction method

This section aims to explain what robust optimal control is, why it is challenging, and the approach behind the local reduction method. This is to help the reader understand the logic behind this approach, which will be formally introduced in Section 6.1.

Robust optimal control aims to find a control input that is resistant to uncertainties, or said in other words, that minimises the deviation from the objective function and boundary constraints b. If we consider an uncertainty that is continuous, and has a lower and upper bound, something becomes clear - there is an infinite amount of values that this uncertainty could take. Therefore, considering uncertainties makes the problem intractable.

One approach would be to reduce the infinite set into a number of unique points in a finite set, similar to what is done in direct collocation for time. This is the basis of all scenario based approaches. The challenging part is how to select the most representative uncertainties, which, as we aim to achieve robustness, would be the most disturbing, or said in other words, the uncertainties that lead to the worse final outcome.

The local reduction method proposes to only consider N_{scen} worst-case scenarios to find a solution, by iterating between the optimal control problem introduced previously (a minimisation optimisation), and a maximisation model(s). This maximisation model looks for the worst case scenario by finding the uncertainty values that would cause the optimal control input to deviate the most from the intended target values. The algorithm can therefore be simplified as follows:

1. Minimisation step: find an optimal control input (u^*) for the finite scenarios considered.
2. Maximisation step: can it find at least one scenario not currently considered that makes u^* deviate from target $\boldsymbol{b}$, or give a higher cost than expected for the objective? If the answer is:
   (a) YES: add this scenario to the set, and repeat Step 1.
   (b) NO: Consider the control input u^* to be *robust* to the given uncertainty.

So to summarise, the proposed method finds what it thinks is a robust solution, proves that it is not and tries again, until it finds a robust solution. If this logic is understood, it will be easier to understand the project and the mathematical formulation introduced in Section 6.1.

So far, it has been shown that for some problems, it can find a smaller number of scenarios that other methods that are more representative of the worst cases, therefore giving a more robust solution at a lower computational cost. This project aims to investigate whether or not this will hold true for a landing scenario.

3 Outline of the project

3.1 Research question

As described in Chapter 2, the local reduction method has been successfully applied to small scale-problems. This project aims to investigate the applicability of the local reduction method to a real lunar landing scenario. Specifically, a lunar landing problem will have a large number of state variables which are multiplied by the number of scenarios, significantly increasing computational complexity. It will also have a large number of highly-complex, non-linear equality and inequality constraints. Additionally, a landing scenario has different sources and types of uncertainties, both time-varying and epistemic. The increase in dimensionality, complexity and number of constraints could therefore prove challenging for the local reduction method, providing an opportunity to test showcase its potential.

The specific research questions are:

1. Is the local reduction method applicable to a lunar landing scenario?
2. What advantages does the local reduction method provide in terms of accuracy compared to current methodologies?
3. How does its computational speed compare to other robust trajectory optimization methods?
4. What types of uncertainties can the local reduction method account for, and can it handle all the uncertainties experienced by a lunar lander?

To answer these questions, several key decisions were taken early on, as justified below.

3.2 Convex vs non-convex model

As explained in Chapter 2, common approaches to lunar landing often include formulating the problem in convex form. Convex form allows for the use of algorithms that converge quickly and ensure global optimality. These are all properties that could help the local reduction algorithm run faster. On the other hand, convexification of a problem is not trivial and would therefore require more work and expertise from the user. Additionally, it shouldn't be assumed that all problems can be convexified loss-lessly.

To avoid loss of generality to other problems, a non-linear, non-convex model was chosen. It will allow for a more realistic model of the problem without simplifiying assumptions, as well as better exemplifying the range of use cases for the local reduction method. If necessary, convexification techniques will be explored, to improve convergence while maintaining solution accuracy.

3.3 Using Julia + JuMP

For the development of the algorithms, Julia combined with the JuMP optimization library were chosen. This combination offers many advantages over other languages like Python or MATLAB. Julia is designed for high-performance numerical computing, providing speed comparable to low-level languages like C and Fortran while maintaining the ease of use of higher-level languages like

Python and MATLAB [18]. In turn, JuMP was chosen for this project due to its performance and flexibility in handling constrained optimization problems. It offers user-friendly syntax that mimics natural mathematical expressions, making it easy to formulate problems with decision variables, objective functions, and constraints. As it is written natively in Julia, it ensures high-speed problem creation, with efficient in-memory communication with solvers. This gives it a speed similar to special-purpose modeling languages such as AMPL. [19]

3.4 Key objectives

In order to answer the research questions proposed, four key objectives were identified:

1. Development and validation of a non-linear, non-convex model.
2. Identification and characterisation of key sources of uncertainty in lunar landing.
3. Development of an implementation of the local reduction method to find robust trajectories.
4. Develop benchmarks to compare robust model to baseline and other methods in the literature.

3.4.1 Development of a non-linear non-convex model

The first milestone for this project should be the development of a nonlinear, non-convex model for lunar landing. Some requirements were set for this objective.

Firstly, the dynamics of the model should accurately reflect the dynamics experienced in the moon, verified by physics equations as well as by comparing against results in the literature. This step was completed with guidance and collaboration from Airbus Defence & Space UK, with flight heritage and state-of-the-art models.

Secondly, said model must show good convergence properties for a range of initial and final conditions. As the minimization optimization is the bottleneck in the min-max model, reducing the number of iterations and testing whether it converges for a range of conditions is the essential first step, and not trivial.

Lastly, the numerical solution was also verified, by checking that the discretisation error for the chosen mesh size is reasonable, and confirming it reaches the global maximum by using a range of starting points.

3.4.2 Identification and characterisation of key sources of uncertainty in lunar landing

Whilst most of this project may be spent in optimisation algorithms and mathematical formulations, the results of the robust trajectory optimisation will only be as good in practice as the characterisation of uncertainty. One key objective of this project should therefore be to identify the main sources of uncertainty, their behaviour relative to the stage of landing, and their bounds.

Additionally, the uncertainties should be ranked in terms of their impact to decide in which order they should be added to the optimisation. The project should analyse ways to model them and whether the local reduction method is a benefit or hindrance for this.

3.4.3 Development of local reduction algorithm to find robust trajectories

To fulfill this objective, the algorithms described in the original paper**ref:kerriganlocrec** should be formulated specifically for the non-linear lunar landing model developed. Special focus should be given to building it in a way that reduces overhead in the algorithm by taking advantage of the features offered by JuMP. The most optimal way of adding scenarios and whether this is different

from smaller models should be discussed, as well as general findings on improving speed of the algorithm.

Another point of focus should be towards its application for a range of objectives, violations and uncertainties. Finally, focus should be given to providing easy-to-understand parameters to give intuitive and detailed control as to the acceptable size of each individual violation.

3.4.4 Benchmark development to compare robust model to baseline

For this final stage, tools shuold be developed to be able to compare the results achieved by the robust and baseline model. The objective should be able to do direct comparisons to the baseline, carry out sensitivity analysis on the bounds of the uncertainties, and do studies on the decrease in landing ellipse vs number of scenarios.

Like in the original paper, it should be shown whether or not the uncertainties found lie on the boundaries or within the set, and therefore draw comparisons between this method and other scenario-based methods.

Finally, allowing for the time-frame of the project, it should be compared to other methods in literature.

4
Development of Nonlinear, Non-Convex Model

4.1 ESA Model

One model considered was based on the preliminary design of ESA's Argonaut lunar lander, and their G&C design [20].

4.1.1 Key Characteristics of Argonaut

Argonaut's D&L preliminary design consisted of a multi-phase optimal control problem as described by Figure 4.1.

The D&L was designed to be separated into 4 stages:

1. A braking burn, where most engines burn at full throttle and against the direction of motion.
2. Pitch-up, where the pitch angle increases to 80 degrees whilst continuing to burn all engines.
3. Main engine cut-off, where only the secondary engines remain operative.
4. Vertical descent & landing, where all engines are switched off and Argonaut slowly falls towards the ground, and only small adjustments from the RCS propellers are carried out.

Additionally, there are two points highlighted in blue, HDA1 and HDA2, where diversion maneuvers can be carried out in case something wrong is detected with the landing zone. For the model design, only two stages were defined: MBB to LGA, with all engines firing, and LGA to VGA, with only two engines firing.

The paper used a set of landing-site referenced spherical coordinates (r, ϕ, θ), so that the positional vector is given by $r = r\left[\cos\phi\cos\theta \quad \sin\phi \quad \cos\phi\sin\theta\right]^\top$. The equations of motion are given

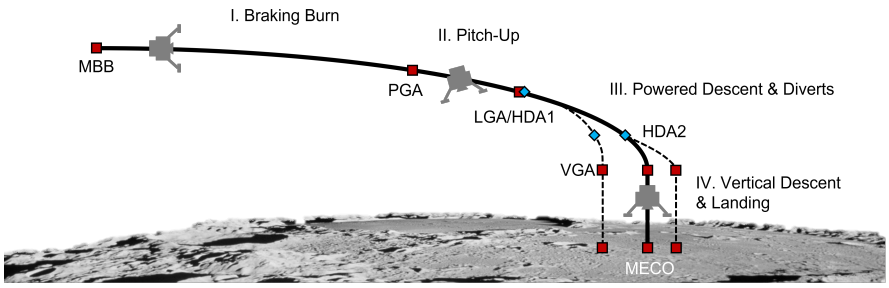

Figure 4.1: Descent & Landing Scenario for Argonaut. This figure is taken from [20].

Table 4.1: Descent and landing parameters and constraints [20].

Parameter	Value	
Max engine thrust	6000	N
Min engine thrust	3000	N
Max engine throttle rate	200	N/s
Engine specific impulse	330	s
Number of engines	3	

Parameter	Value	
Wet mass at MBB	7000	kg
Max steering rate	5	deg/s
VGA altitude	30	m
Vertical descent pitch angle	90	deg
Vertical descent velocity	2	m/s

in equation 4.1.

$$\dot{r} = v_r \tag{4.1a}$$

$$\dot{\phi} = \frac{v_\phi}{r} \tag{4.1b}$$

$$\dot{\theta} = \frac{v_\theta}{r\cos\phi} \tag{4.1c}$$

$$\dot{v}_r = \frac{v_\phi^2 + v_\theta^2}{r} - \frac{\mu}{r^2} + \frac{T_1 + T_2}{m}\cos\alpha\cos\beta \tag{4.1d}$$

$$\dot{v}_\phi = -\frac{v_\phi v_r}{r} - \frac{v_\theta^2}{r}\tan\phi + \frac{T_1 + T_2}{m}\sin\alpha \tag{4.1e}$$

$$\dot{v}_\theta = -\frac{v_\theta v_r}{r} + \frac{v_\phi v_\theta}{r}\tan\phi + \frac{T_1 + T_2}{m}\cos\alpha\sin\beta \tag{4.1f}$$

$$\dot{m} = -\frac{T_1 + T_2}{I_s g_0} \tag{4.1g}$$

where v_r, v_ϕ, and v_θ are the projections of the inertial velocity vector in the (e_r, e_ϕ, e_θ) reference frame, whilst α and β are the azimuth and elevation angles respectively, constraining the thrust vector direction.

One simplification of this model is that there is no equation of motion for the thrust vector direction, instead defined by variables that are constrained by the maximum pitch rate, from Table 4.1. The model assumes that this pitch rate will be generated by a set of RCS thrusters, for which it neglects the mass exerted. T_1 is the thrust from the two outer engines, and T_2 is the thrust of the main engine, which will switch off at LGA.

4.1.2 Optimal Control Problem Formulation

Using the equations above and information from the tables, a multi-phase optimal control problem can be formulated in standard form as shown in Problem 1. The decision variables include the magnitude of the thrusts, their direction, the final time, and the time at which the main engine is switched off. Aside from the boundary conditions, all of the equality and inequality constraints are applied for the whole time frame. The only exception is for T_2 to account for Central Engine Cut-Off (CECO). To make it tractable, the direct collocation method is applied; variables are discretised at 'n' collocation points, at which the constraints are satisfied, whilst the values in between are interpolated by splines. This means that for some variable $x(t) \to x \in \mathbb{R}^n$.

Trapeizodal collocation was chosen at this stage for simplicity, which can be changed for other methods such as Gauss-Legendre quadrature. The pseudo-code from the discretised model can be seen in Code 4.1. This code has been simplified for clarity by removing the boundary conditions, to better highlight the changes due to discretisation.

Problem 1 *Multi-phase Argonaut Model*

Cost Function:

$$\underset{\substack{T_1,T_2,\alpha,\\ \beta,t_f,t_{\text{CECO}}}}{\text{minimize}} \quad -m(t_f)$$

s.t

Dynamics:

$$\dot{r}(t) = v_r(t)$$
$$\dot{\phi}(t) = \frac{v_\phi}{r}$$
$$\dot{\theta}(t) = \frac{v_\theta(t)}{r(t)\cos\phi(t)}$$
$$\dot{v}_r(t) = \frac{v_\phi(t)^2 + v_\theta(t)^2}{r(t)} - \frac{\mu}{r(t)^2} + \frac{T_1(t)+T_2(t)}{m}\cos\alpha(t)\cos\beta(t)$$
$$\dot{v}_\phi(t) = -\frac{v_\phi(t)v_r(t)}{r(t)} - \frac{v(t)_\theta^2}{r(t)}\tan\phi + \frac{T_1(t)+T_2(t)}{m(t)}\sin\alpha(t)$$
$$\dot{v}_\theta(t) = -\frac{v_\theta(t)v_r(t)}{r(t)} + \frac{v_\phi(t)v_\theta(t)}{r(t)}\tan\phi(t) + \frac{T_1(t)+T_2(t)}{m(t)}\cos\alpha(t)\sin\beta(t)$$
$$\dot{m}(t) = -\frac{T_1(t)+T_2(t)}{I_s g_0}, \qquad \forall t \in (0, t_f)$$

Boundary Conditions:

$$r(t_0) = R_{\text{Moon}} + 30\,\text{km}, \quad r(t_f) = R_{\text{Moon}} + 30\,\text{m}$$
$$v_\theta(t_0) = 1681.6 \quad v_\theta(t_f) = 0$$
$$\phi(t_0) = \phi(t_f) = v_\phi(t_0) = v_\phi(t_f) = 0$$
$$m(t_0) = 0$$
$$\theta(t_f) = \frac{\pi}{2}$$

State Constraints:

$$r(t_f) \le r, \quad \forall t \in (0, t_f)$$

Control Constraints:

$$6\,\text{kN} \le T_1(t) \le 12\,\text{kN}$$
$$3\,\text{kN} \le T_2(t) \le 6\,\text{kN} \quad \forall t \in (0, t_{\text{CECO}}]$$
$$T_2(t) = 0 \quad \forall t \in (t_{\text{CECO}}, t_f)$$
$$\alpha \in (0, 2\pi), \quad \beta \in (0, \pi)$$
$$|\dot{\alpha}| \le 5\,\text{deg}/s, \quad \left|\dot{\beta}\right| \le 5\,\text{deg}/s$$

Listing 4.1: Pseudo-code showing how the continous problem may be discretised in Julia.

```
@objective(Max, m[n])

@variables(
    #State variables
    r₃ ≤ r[1:n]                 # m
    ϕ[1:n]                      # rad
    θ[1:n]                      # rad
        ⋮$\vdots$
    Δt[1:n]
    #Control variables
    T_min*2 ≤ T₁[1:n] ≤ T_max*2
    T₂[1:n]
    0 ≤ α[1:n] ≤ 2π
    0 ≤ β[1:n] ≤ π
)
 j  2:n
```

```
    i = j - 1
    @constraints(
        #Dynamic constraints
        r[j] == r[i] + 0.5 * Δt[i] * (δr[j] + δr[i])
        ϕ[j] == ϕ[i] + 0.5 * Δt[i] * (δϕ[j] + δϕ[i])
                                ⋮
        v_θ[j] == v_θ[i] + 0.5 * Δt[i] * (δv_θ[j] + δv_θ[i])
        m[j] == m[i] + 0.5 * Δt[i] * (δm[j] + δm[i])
        )
        #Control constraints
        -2*Tdot_max≤ (T₁[j] - T₁[i])/Δt[i] ≤ 2*Tdot_max
        -max_pitchrate ≤ (α[j] - α[i])/Δt[i] ≤ max_pitchrate
        -max_pitchrate ≤ (β[j] - β[i])/Δt[i]  ≤ max_pitchrate
        )


j  2:n_MECO
    @constraint(
        ⋮
```

Note that $\delta x[j]$ is the equivalent of $\dot{x}(t[j])$, so for example `δvᵣ[j] = (v_θ[j]^2 + v_ϕ[j]^2)/r[j] - μ/r[j]^2 + (T₁[j] + T₂[j])/m[j]*cos(α[j])*cos(β[j]))`.

In this version, t_{CECO} is defined based on a given mesh number n_{CECO}, after which the constraints on T_2 change. Δt allows the time between collocation points to vary, so that the optimization algorithm decides on the positioning of t_{CECO} freely.

To reduce the number of variables, a version where Δt only has one value per stage was made. The final iteration of two stage model was further improved by reducing the size of T_2 from $\mathbb{R}^n$ to $\mathbb{R}^{n_{MECO}}$, and not including T_2 in the dynamic equations for the second stage, eliminating the fixed, unused variables. Finally, a single stage optimisation problem was defined for the diversion maneuver described above. This model only consisted of the second stage with different initial and final conditions.

Once discretized, the optimization problem was solved using an Interior Point Method, through its Julia `Ipopt.jl` wrapper.

4.1.3 Verification

The model was validated by analyzing the optimal solution, for any indications of potential errors in the physical model, its discretised form, or the constraint formulation. Results for the initial conditions provided in the original paper are presented in Figure 4.2.

Figure 4.2 displays all the variables in the model. The graphs are generally smooth and exhibit trajectory motion, suggesting that the physical constraints are effectively applied. However, a notable exception occurs in the α and β values, where sharp changes in their gradients are observed. This is expected, as the model only limits the maximum and minimum gradients without accounting for dynamic effects. Given the vehicle's weight, it is unlikely that the RCS thrusters could counteract the moment of inertia so effectively, indicating a potential area for model improvement.

To calculate the error introduced by the collocation, it was estimated using the following equation for discretisation error. [17]

$$\epsilon(t) = \dot{x}(t) - f(t, x(t), u(t)) \tag{4.2}$$

As can be observed in figure 4.3, the discretization error is zero at the node points, as expected, and increases as it moves further away. These values can be integrated by:

$$\eta_k = \int_{t_k}^{t_{k+1}} |\epsilon(\tau)| \, d\tau \tag{4.3}$$

To give the right hand side of the figure. The values were integrated using `quadgk()` which provides

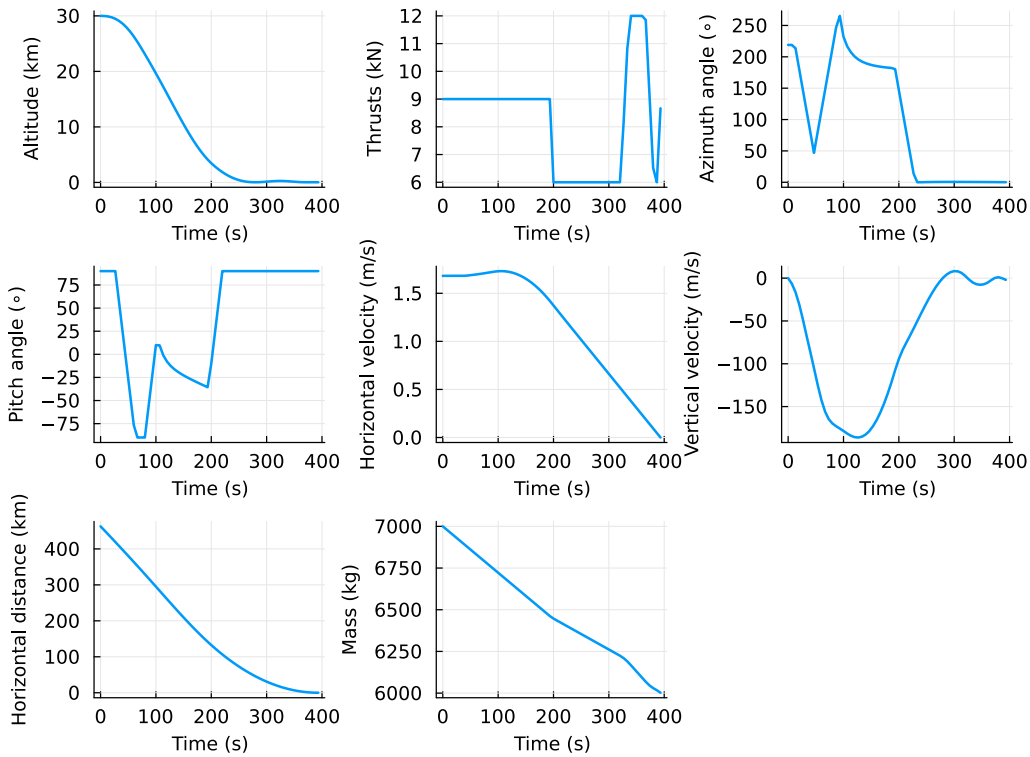

Figure 4.2: Trajectory from original model shown.

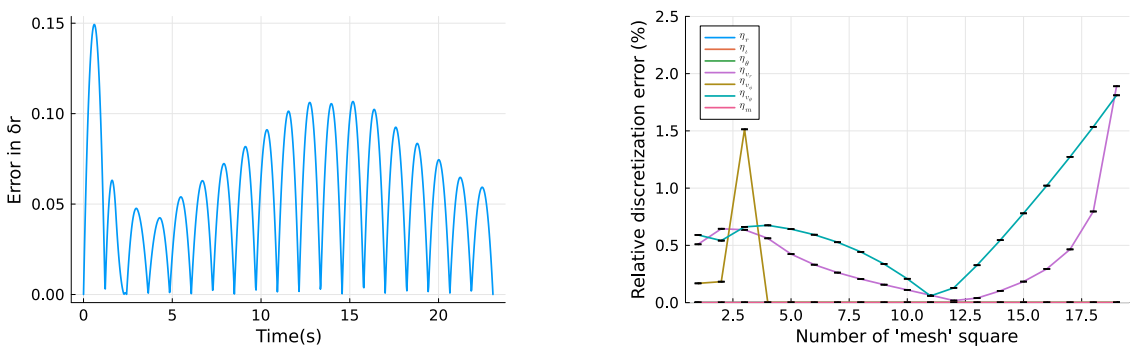

Figure 4.3: Discretization error i) absolute error for altitude, ii) relative error per timestep for all variables

Table 4.2: Average discretisation error per timestep for each variable

Element	Average error per timestep (%)
r	3.590e-6
θ	1.338e-9
ϕ	2.294e-7
v_r	0.3848
v_ϕ	0.09351
v_θ	0.6350
m	6.8242e-18

a measure of the integration error, plotted as error bars, albeit the values are very small and hard to see. The integration error was summed to the discretisation error to provide a more accurate estimate of the total discretisation error in table 4.2.

For the velocities, these values are significant. Therefore the resolution of the mesh should be increased or a different method of collocation used. As this model was abandoned this was not implemented.

4.1.4 Limitations of the model

Problems with the formulation became apparent when optimising for the diversion maneuver, which initially seemed unfeasible. This was identified to be due to the angle constraint, as defined in the pseudocode 4.1.

As 90° is the maximum defined for the elevation, the algorithm instead rotates around 180° to achieve the desired position. As each angle is limited by $5°/s$ per specifications, this rotation takes too long (36 seconds) when in reality it could be done in 2 seconds. When the constraint on maximum pitch angle was removed, the problem became feasible.

However, this optimal problem is no longer a fair representation of the real landing scenario, and therefore would not be able to prove whether or not local reduction is applicable, so an alternative formulation was sought, as described in section 4.2.

4.2 Quaternion Formulation

Due to the problems with the original formulation, a formulation that used linear algebra and quaternions for the rotational system was sought. This was inspired by [21].

4.2.1 Differences between the models

As described by the model name, the main difference between the model is its move towards quaternion transformations between two frames of reference:

(a) $\mathcal{F}_\mathcal{I}$: An inertially-fixed Up-East-North reference frame with its origin located at the landing site.
(b) $\mathcal{F}_\mathcal{B}$: A body-fixed frame centered at the vehicle center-of-mass, with its X-axis pointing along the vertical axis of the vehicle, its Y-axis pointing out the side of the vehicle, and its Z-axis completing the right-handed system.

The thrust model was changed to gimballed thrust, so that the only control input now becomes $T_\mathcal{B}(t) \in \mathbb{R}^3$. This is more challenging from a control point of view, and therefore arguably a better case study. Thanks to the quaternion formulation, it is very easy to change this for main thrust and RCS thrusters or other combinations.

In addition, the rotational dynamics are fully defined. The optimization model has been changed to a single optimization stage to focus on the diversion maneuver. This is because it was desired to begin with a single-phase optimization problem, and the diversion maneuver is the most challenging, giving the most room for deviations from the target.

The new dynamic equations are:

$$\dot{\mathbf{r}}_{\mathcal{I}}(t) = \mathbf{v}_{\mathcal{I}}(t) \tag{4.4a}$$

$$\dot{\mathbf{v}}_{\mathcal{I}}(t) = \frac{1}{m(t)}\mathbf{F}_{\mathcal{I}}(t) + \mathbf{g}_{\mathcal{I}} \tag{4.4b}$$

$$\dot{q}_{\mathcal{B}/\mathcal{I}}(t) = \frac{1}{2}\Omega\big(\boldsymbol{\omega}_{\mathcal{B}}(t)\big)q_{\mathcal{B}/\mathcal{I}}(t) \tag{4.4c}$$

$$J_{\mathcal{B}}\dot{\boldsymbol{\omega}}_{\mathcal{B}}(t) = \mathbf{M}_{\mathcal{B}}(t) - [\boldsymbol{\omega}_{\mathcal{B}}(t)\times]\, J_{\mathcal{B}}\boldsymbol{\omega}_{\mathcal{B}}(t) \tag{4.4d}$$

$$\dot{m}(t) = \alpha_{\dot{m}}\|\mathbf{T}_{\mathcal{B}}(t)\|_2 \tag{4.4e}$$

where $\mathbf{r}_{\mathcal{I}}(t) \in \mathbb{R}^3$, $\mathbf{v}_{\mathcal{I}}(t) \in \mathbb{R}^3$, and $\mathbf{F}_{\mathcal{I}}(t) \in \mathbb{R}^3$ are defined in the inertial frame, and where $q_{\mathcal{B}/\mathcal{I}}(t) \in \mathcal{S}^3$, $\boldsymbol{\omega}_{\mathcal{B}}(t) \in \mathbb{R}^3$, are defined from the body frame relative to the inertial frame. $\mathbf{M}_{\mathcal{B}}(t) \in \mathbb{R}^3$ denotes the torque acting on the vehicle. The unit quaternions have been defined using the Hamiltonian form (scalar-last notation), for ease of use with Julia's 1-based array indexing, so that:

$$q_{\mathcal{B}/\mathcal{I}} = \begin{bmatrix}\sin(\xi/2)\hat{\mathbf{n}} \\ \cos(\xi/2)\end{bmatrix} = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 \end{bmatrix}^T$$

and so:

$$C_{\mathcal{B}/\mathcal{I}} = \begin{bmatrix} 1-2(q_2^2+q_3^2) & 2(q_1q_2+q_4q_3) & 2(q_1q_3-q_4q_2) \\ 2(q_1q_2-q_4q_3) & 1-2(q_1^2+q_3^2) & 2(q_2q_3+q_4q_1) \\ 2(q_1q_3+q_4q_2) & 2(q_2q_3-q_4q_1) & 1-2(q_1^2+q_2^2) \end{bmatrix}.$$

The quaternion derivative in terms of angular velocity can be expressed for some $\boldsymbol{\xi} \in \mathbb{R}^3$ as $\Omega(\boldsymbol{\xi})$:

$$\Omega(\boldsymbol{\xi}) = \begin{bmatrix} 0 & -\xi_x & -\xi_y & -\xi_z \\ \xi_x & 0 & \xi_z & -\xi_y \\ \xi_y & -\xi_z & 0 & \xi_x \\ \xi_z & \xi_y & -\xi_x & 0 \end{bmatrix}.$$

Finally, the force and torque vectors, $\mathbf{F}_{\mathcal{I}}(t)$ and $\mathbf{M}_{\mathcal{B}}(t)$, may be related to the thrust vector $\mathbf{T}_{\mathcal{B}}(t)$ by:

$$\mathbf{F}_{\mathcal{I}}(t) = C_{\mathcal{I}/\mathcal{B}}(t)\mathbf{T}_{\mathcal{B}}(t) \tag{4.5a}$$

$$\mathbf{M}_{\mathcal{B}}(t) = [\mathbf{r}_{T,\mathcal{B}}\times]\,\mathbf{T}_{\mathcal{B}}(t). \tag{4.5b}$$

4.2.2 Optimal control formulation

As done for the previous section, the dynamics can be put into a continous time optimization problem, as shown in Problem 2. The state constraints remain the same albeit reformulated: whilst previously it was constraining the decision variables α and β, it now constraining the overall angular velocity. In terms of thrust constraints, the magnitude constraint remains the same. The last constraint limits the maximum angle from the axial direction. This model assumes that thrust vectoring happens quickly enough to be considered instantaneous. The model also assumes that burning fuel does not change the moment of inertia or weight distribution.

Problem 2 *Quaternion-Based Model*

Cost Function:

$$\underset{t_f, \mathbf{T}_{\mathcal{B}}(t)}{\text{minimize}} \quad -m(t_f)$$

$$\text{s.t, } \forall t \in (0, t_f)$$

Dynamics:

$$\dot{m}(t) = -\alpha_{\dot{m}} \|\mathbf{T}_{\mathcal{B}}(t)\|_2$$
$$\dot{\mathbf{r}}_{\mathcal{I}}(t) = \mathbf{v}_{\mathcal{I}}(t)$$
$$\dot{\mathbf{v}}_{\mathcal{I}}(t) = \frac{1}{m(t)} C_{\mathcal{I}/\mathcal{B}}(t) \mathbf{T}_{\mathcal{B}}(t) + \mathbf{g}_{\mathcal{I}}$$
$$\dot{q}_{\mathcal{B}/\mathcal{I}}(t) = \frac{1}{2} \Omega\big(\boldsymbol{\omega}_{\mathcal{B}}(t)\big) q_{\mathcal{B}/\mathcal{I}}(t)$$
$$J_{\mathcal{B}} \dot{\boldsymbol{\omega}}_{\mathcal{B}}(t) = [\mathbf{r}_{T,\mathcal{B}} \times] \, \mathbf{T}_{\mathcal{B}}(t) - [\boldsymbol{\omega}_{\mathcal{B}}(t) \times] \, J_{\mathcal{B}} \boldsymbol{\omega}_{\mathcal{B}}(t)$$

Boundary Conditions:

$$m(0) = m_{wet}$$
$$\mathbf{r}_{\mathcal{I}}(0) = \mathbf{r}_{\mathcal{I},i} \qquad \mathbf{r}_{\mathcal{I}}(t_f) = \mathbf{0}$$
$$\mathbf{v}_{\mathcal{I}}(0) = \mathbf{v}_{\mathcal{I},i} \qquad \mathbf{v}_{\mathcal{I}}(t_f) = \mathbf{v}_{\mathcal{I},f}$$
$$q_{\mathcal{B}/\mathcal{I}}(t_f) = q_{\mathcal{B}/\mathcal{I},f}$$
$$\boldsymbol{\omega}_{\mathcal{B}}(0) = \boldsymbol{\omega}_{\mathcal{B},i} \qquad \boldsymbol{\omega}_{\mathcal{B}}(t_f) = \mathbf{0}$$
$$\mathbf{e}_2 \cdot \mathbf{T}_{\mathcal{B}}(t_f) = \mathbf{e}_3 \cdot \mathbf{T}_{\mathcal{B}}(t_f) = 0$$

State Constraints:

$$\|\boldsymbol{\omega}_{\mathcal{B}}(t)\|_2 \leq \omega_{max}$$
$$\mathbf{e}_1 \cdot \mathbf{r}_{\mathcal{I},f} \leq \mathbf{e}_1 \cdot \mathbf{r}_{\mathcal{I}}(t)$$

Control Constraints:

$$T_{min} \leq \|\mathbf{T}_{\mathcal{B}}(t)\|_2 \leq T_{max}$$
$$\cos \delta_{max} \|\mathbf{T}_{\mathcal{B}}(t)\|_2 \leq \mathbf{e}_1 \cdot \mathbf{T}_{\mathcal{B}}(t)$$

4.2.3 Results & Verification

This model was optimized using Ipopt with the parameters seen in Table 4.3. Whilst initially the same conditions as the first model were used, a more aggressive diversion maneuver was defined, where there is an initial velocity causing it to move away from the target point. This has been chosen as it is considered one of the hardest control maneuvers in powered descent.

The resulting trajectory can be observed in Figure 4.4.

To better understand the orientation, the spacecraft has been plotted as a 3D pyramid, with the thrust direction being shown by a colored line. From observation, we can verify that the dynamics make sense: the spacecraft firstly turns around to fire against the extra velocity inparted, before turning around in the opposite direction to remove the out of plane velocities created in the turning.

The discretization error was calculated in a similar fashion to how it is calculated above. The sum of the values are displayed in table 4.4.

Various initial conditions were tried, and unlike with the first model, no problems were found with this one. Because of this, it will be used as the basis on which to apply the local reduction method. Additionally, it was tested for a range of starting values to prove that it converged to the global optimum. For the final time, this can be observed in Figure 4.5. Note that these values are normalised, so that

Table 4.3: Conditions used for second model

Parameter	Value	
Mass	2000	kg
Max engine thrust	5000	N
Min engine thrust	100	N
$\alpha_{\dot{m}}$	3.09e-4	kg/N
δ_{max}	90	deg

Parameter	Value	
$\mathbf{r}_{\mathcal{I},i}$	[400 400 0]	m
$\mathbf{r}_{\mathcal{I},f}$	[0 0 0]	m
$\mathbf{v}_{\mathcal{I},i}$	[0 10 0]	m/s
$\mathbf{v}_{\mathcal{I},f}$	[-3.68 0 0]	m/s
$\boldsymbol{\omega}_{\mathcal{B}}$	[0 0 0]	m/s
$q_{\mathcal{B}/\mathcal{I}}$	[1 0 0 0]	m/s

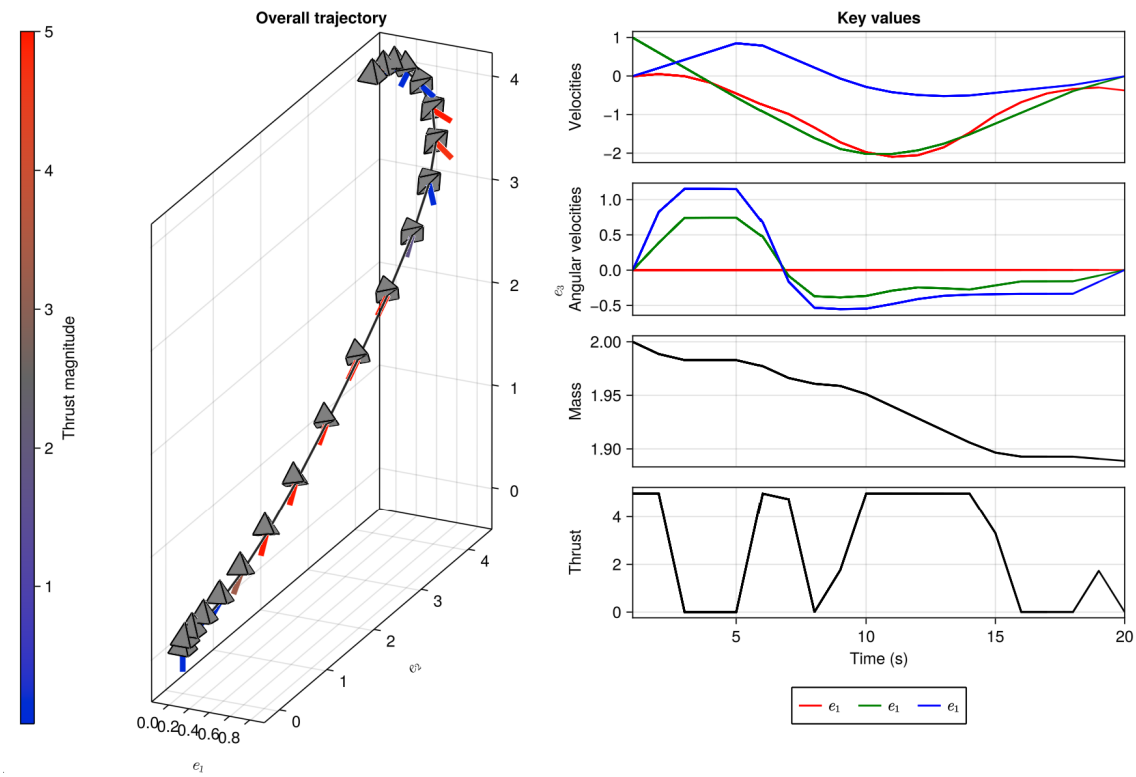

Figure 4.4: Initial trajectory observed for Problem 2

Table 4.4: Average discretization error per timestep for each variable

Element	Average error per timestep (%)
$\mathbf{r}_{\mathcal{I},1}$	4.560e-7
$\mathbf{r}_{\mathcal{I},2}$	2.760e-7
$\mathbf{r}_{\mathcal{I},3}$	9.870e-8
$\mathbf{v}_{\mathcal{I},1}$	2.304
$\mathbf{v}_{\mathcal{I},2}$	1.752
$\mathbf{v}_{\mathcal{I},3}$	3.402
$q_{\mathcal{B}/\mathcal{I},1}$	1.347e-4
$q_{\mathcal{B}/\mathcal{I},2}$	2.647e-6
$q_{\mathcal{B}/\mathcal{I},3}$	3.238e-7
$q_{\mathcal{B}/\mathcal{I},4}$	1.338e-9
$\boldsymbol{\omega}_{\mathcal{B},1}$	0.3151
$\boldsymbol{\omega}_{\mathcal{B},2}$	0.0935
$\boldsymbol{\omega}_{\mathcal{B},3}$	0.2051
m	8.467e-12

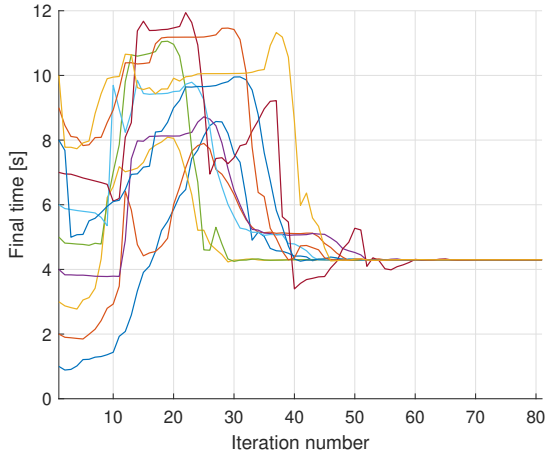

Figure 4.5: Different initialisations for the final time, all converging towards 4.29s .

4.2.4 Hazards to avoid

There were initially many difficulties implementing this model, this section summarizes the main difficulties and how to overcome them.

The first hurdle is that JuMP does not support linear algebra by itself, and therefore all functions must be defined manually, for example, for some $\boldsymbol{\xi} \in \mathbb{R}^3$:

$$[\boldsymbol{\xi}\times] = \begin{bmatrix} 0 & -\xi_z & \xi_y \\ \xi_z & 0 & -\xi_x \\ -\xi_y & \xi_x & 0 \end{bmatrix}$$

To implement this, it is best to define linear algebra functions as usual Julia functions (not JuMP expressions) and then call them with each variable, so that JuMP interprets the calculation as a linear combination of variables. Code 4.2 demonstrates this.

Listing 4.2: Linear algebra in JuMP example

```
 Omega(ξ)
     [
        0    -ξ[1] -ξ[2] -ξ[3]
        ξ[1]     0  ξ[3] -ξ[2]
        ξ[2] -ξ[3]     0  ξ[2]
        ξ[3]  ξ[2] -ξ[1]    0
    ]

ξcross(ξ) = [0 -ξ[3] ξ[2]; ξ[3] 0 -ξ[1]; -ξ[2] ξ[1] 0]
model = Model(Ipopt.Optimizer)
@expressions(model,
    Δτ, t_f / (n-1)
    δm[j  1:n], -αₘ * sqrt(sum(T_B[j, :].^2))
    δv[j  1:n], g .+ C_IB_func(q[j, :]) * T_B[j, :] ./ m[j]
    δq[j  1:n], 0.5 * Omega(ω[j, :]) * q[j, :]#./sqrt(sum(q[j, :].^2))
    δω[j  1:n],
    inv(J_B) * (ξcross(r_TB)*T_B[j, :] - ξcross(ω[j, :])*J_B*ω[j, :])
)
```

Another point of trouble was the quaternions. To make sure that they are unit quaternions, the constraint $\|q_{\mathcal{B}/\mathcal{I}}\|_2 = 1$ may be added. However, adding this constraint greatly increases the amount of iterations taken to achieve convergence; from 80 to about 800. This is because the nonlinear equality constraint forces the quaternion to lie in the unit sphere. To solve this, many

algorithms will project the updated $q_{\mathcal{B}/\mathcal{I}}$ back onto the constraint manifold after each iteration. Not only does this increase the amount of computations, but it may also reduce the effective step size when it is projected onto the tangent space, resulting in smaller steps towards the optimal solution.

The best strategy to overcome this is to not include the unit quaternion constraint at all - if your boundary conditions fix the initial value to be within the 3-sphere and your dynamic constraints are well defined, the quaternions should fulfill the constraint automatically. However, this should not be taken for granted, as either due to discretization error or something else going wrong it may deviate. It is therefore always good to calculate the norm as a sanity check, and if significant deviations are found, to use reprojection instead.

Finally, for some use cases, the thrust constraint, especially when combined with the quaternion constraint, caused issues. One method that fixed this was to instead define it as a magnitude and two angles. Whilst this brings back the bounds on the angle are less problematic as there will never be 360 degrees turns. It can still be problematic when it comes to rotations around the axial direction. Therefore, in the last iteration of this model the original form was used.

5 Uncertainty characterisation

5.1 Key sources of uncertainty

As mentioned in previous sections, the method will only be as good as the characterisation of the uncertainties. To understand the potential sources of uncertainty, it is important to understand the kind of sensors used in lunar landing missions. For the final stages of landing considered, the following sensors are expected to be used.

Table 5.1: Table showing the sensors used for attitude estimation with the usual level of accuracy that can be expected. [22]

Sensor	Operational constraints (m)	Level of accuracy
Terrain Relative Navigator (TRN)	7000-10	10-100m
Navigation Doppler LIDAR (NDL)	4000-30	0.02m/s
Inertial Measurement Unit	Whole range	1°/hr 0.001g

Table 5.1 shows how for TRN, the accuracy varies with altitude, with increasing accuracy with decreasing altitude. TRN gives measurements of the position above a planet, and therefore gives a measure for the error expected for $\mathbf{r}_{\mathcal{I},2}$ and $\mathbf{r}_{\mathcal{I},3}$. TRN is a technology that compares the terrain below the lander with a database, giving it a position in a map. For higher accuracy, a higher pixel density will be needed to be analysed. Because of this, pixel density is kept the same so that a more accurate value can be achieved the closer it gets to the surface. At altitudes of less than 30m, there are not enough unique characteristics that can be observed to place it in a given position. It can give accurate measurements until very low altitudes past the point of unpowered descent, and therefore can be considered to remain on for the duration of the mission being considered.

The NDL sensor can in turn provide very accurate ranging and velocity relative to the surface, giving good altitude measures. However, it is limited in height, and not available for the lower altitudes. It can be coupled with a dedicated low-altitude altimeter. Results for a Monte Carlo simulation to estimate altitude error may be observed in Figure 5.1. At lower altitudes, a significant increase in altitude error up to ± 0.5m is observed.

Aside from sensors, the most significant source of uncertainty that can be observed for lunar landing is given by the thrusters. Rocket engine thrust variation typically falls within a 1-2% range, a variability that is influenced by numerous factors such as combustion efficiency, injector design, and control system precision. This can be reduced with closed-loop feedback systems to about a 0.5-1% range [23].

5.2 Compatibility with local reduction

This section will discuss the ways that uncertainties can be characterised for the local reduction method.

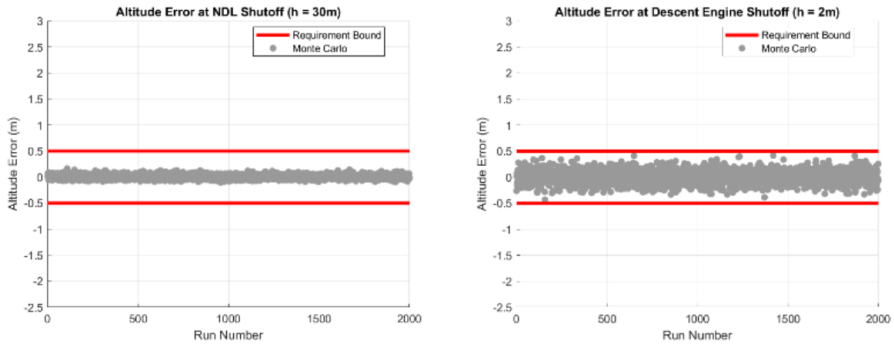

Figure 5.1: Altitude error before and after switch-off of NDL. This figure is taken from [22]

5.2.1 Distributions

The first point to cover is distributions. The way the local reduction method has been implemented enforces hard bounds on the uncertainties, and assumes the uncertainties follow a random distribution within those bounds. Whilst hard to see the point density in Figure 5.1, it seems like this model might fit the uncertainties observed well.

Other common distributions used to model uncertainties may be Gaussian distributions, as many times, most oscillations happen close to the nominal point and only some happen further away. It is therefore arguable that a Gaussian distribution may better reflect what is experienced physically. It is also possible that it is hard to define a hard bound that includes all values.

However, given the nature of this methodology, which is to provide a robust method, it is better to have a conservative estimate, so having hard bounds for this method may be a feature more than a nuisance. Further on, a good model must always be falsifiable, and this expands to uncertainty models. A random distribution within a given bound is very easy to falsify - it is falsified as soon as a value outside of the bounds is encountered. Therefore, when considering robustness and validation, it is the better choice.

There still remains the question that for variance from the nominal case, it might be difficult to set the right bound. Because of this, sensitivity analysis will be carried out to see how much it deviates when uncertainties are higher than expected.

5.2.2 Uncertainty as a function of altitude/time

In the previous section, it has been observed that uncertainty sizes vary with altitude.

For the altitude, it is observed that at the switch-off point for the NDL, the estimation error increases to ± 0.5m. For the local reduction method, this could be modelled by having a multi-phase problem, similar to what was done in Chapter 4 initially. For the error in the TRN, the bounds could potentially be defined as:

$$f_l(\mathbf{r}_{\mathcal{I}}, 1) \leq d_{\mathbf{r}_{\mathcal{I}},1} \leq f_u(\mathbf{r}_{\mathcal{I}}, 1). \tag{5.1}$$

Whilst in this case, it would only affect the initial measurement (when the initial state is taken), it could potentially be expanded for time-varying uncertainties.

Due to the proof-of-concept nature of the project, it was decided to only include constant bounds for now. However, it is likely that the method can be expanded to include this type of uncertainties.

Table 5.2: Uncertainties to include in the model

Order	Name	Type	Magnitude
1	$w_{\mathbf{T}_{\mathcal{B}}}$	Time-varying	$\pm 0.5\%$
2	$d_{\mathbf{r}_{\mathcal{I},1}}$	Initial	± 0.5m
3	$d_{\mathbf{r}_{\mathcal{I},2}}$	Initial	± 5m

5.3 Chosen uncertainties to include in the model

Based on the previous sections, the uncertainties to add, in the given order, are shown in Table 5.2. This order was chosen based on the potential impact of the uncertainty on the mission and interest for the development of the local reduction method.

The uncertainty $w_{\mathbf{T}_{\mathcal{B}}}$ varies in time and is therefore traditionally more difficult to deal with, and therefore is a better showcase of the local reduction method. Additionally, it relates directly to the control input, and therefore something that the method can better optimise to mitigate. Afterwards, the initial uncertainty on altitude was chosen. This is because altitude accuracy is generally more important than sideways accuracy.

6
Local reduction development

After a proven non-linear, non-convex model was developed with good convergence capabilities, a working implementation of the local reduction for this method was developed. This section explains the current formulation, the development process and refinements made, prove of convergence and general guidance on implementing the method in JuMP.

6.1 The local reduction algorithm

The implementation developed can be found in algorithm 1. In the algorithm, x is shorthand for the states $[\mathbf{r}_{\mathcal{I}}, \mathbf{v}_{\mathcal{I}}, q_{\mathcal{B}/\mathcal{I}}, \boldsymbol{\omega}_{\mathcal{B}}, m]$, $w_{\mathbf{T}_{\mathcal{B}}}$ refers to the time-varying uncertainty in the thrust magnitude. In the implementation shown, this is the only uncertainty included, as it was identified as the disturbance with the highest potential impact in Chapter 5.

Algorithm 1: Local reduction implementation

Input: Initial solution for $\mathbf{T}_{\mathcal{B}}$, $\mathbf{x}$, γ and $\mathbb{H}_1 \neq \emptyset$
Output: Optimal $\mathbf{T}^*_{\mathcal{B}}$, $\mathbf{x}^*$, γ^*, set of scenarios $\mathbb{H}^*$

Set $\mathbf{T}^1_{\mathcal{B}} \leftarrow \mathbf{T}_{\mathcal{B}}$, $\mathbf{x}^1 \leftarrow \mathbf{x}$, $\gamma^1 \leftarrow \gamma$, $j \leftarrow 1$
repeat
  forall $k \in 1, 2, ..., n_{max}$ **do**
    Compute $G^k_{\max}(\mathbf{T}^j_{\mathcal{B}}, \mathbf{x}^j, \gamma^j, \mathbb{W}^N \times \mathbb{D})$ and a maximizer $(\mathbf{x}^j, \mathbf{T}_{\mathcal{B}}{}^j, \mathbf{w}_{\mathbf{T}_{\mathcal{B}}})$ by solving Problem 4 with $\mathbb{H} = \mathbb{W}^N \times \mathbb{D}$.
  end
  if $\exists\ G_{\max}(\mathbf{T}^j_{\mathcal{B}}, \mathbf{x}^j, \gamma^j, \mathbb{W}^N \times \mathbb{D}) \geq \epsilon_{tol}$ **then**
    Add new scenario

$$\mathbb{H}_{j+1} \leftarrow \mathbb{H}_j \cup \left\{ (\mathbf{w}^{\mathbf{j},\mathbf{k}}_{\mathbf{T}_{\mathcal{B}}} \mid \mathbf{k} \text{ s.t. } \mathbf{G}^{\mathbf{k}}_{\max}(\mathbf{T}^{\mathbf{j}}_{\mathcal{B}}, \mathbf{x}^{\mathbf{j}}, \gamma^{\mathbf{j}}, \mathbb{W}^{\mathbf{N}} \times \mathbb{D}) = \max\left(\mathbf{G}^{\mathbf{k}}_{\max}(\mathbf{T}^{\mathbf{j}}_{\mathcal{B}}, \mathbf{x}^{\mathbf{j}}, \gamma^{\mathbf{j}}, \mathbb{W}^{\mathbf{N}} \times \mathbb{D})\right) \right\}$$

    Find a $(\mathbf{T}^{j+1}_{\mathcal{B}}, \mathbf{x}^{j+1}, \gamma^{j+1})$ that solves Problem 3 with $\mathbb{H}_{j+1}$
  else

$$\mathbb{H}_{j+1} \leftarrow \mathbb{H}_j$$

  end
  Set $j \leftarrow j + 1$
  Set $(\mathbf{T}^*_{\mathcal{B}}, \mathbf{x}^*, \gamma^*) \leftarrow (\mathbf{T}^j_{\mathcal{B}}, \mathbf{x}^j, \gamma^j)$ and $\mathbb{H}^* \leftarrow \mathbb{H}_j$.
until $\operatorname{card} \mathbb{H}_j = \operatorname{card} \mathbb{H}_{j-1}$;

The way this initial model works can be summarised as follows. Initially, an optimal trajectory for the min model formulated in Problem 2 with one scenario is solved. This scenario can be chosen randomly or for any given values. The optimal $\mathbf{T}_{\mathcal{B}}$ and expected cost function value γ calculated here is inputted into the maximisation models ($\mathbf{T}^1_{\mathcal{B}} \leftarrow \mathbf{T}_{\mathcal{B}}$, $\mathbf{x}^1 \leftarrow \mathbf{x}$, $\gamma^1 \leftarrow \gamma$, $j \leftarrow 1$). The maximisation models looks for the values $\mathbf{w}_{\mathbf{T}_{\mathcal{B}}}$ that maximise $G_{max} = \gamma - m_f$. As γ is input from the minimisation model, this is effectively looking for a worse (lower) final mass than the

minimisation model has predicted, and for the biggest deviation from the final state values for other maximisation steps. If for any of the maximisation models the objective value is positive, and therefore there is a significant deviation ($G_{max} \geq \epsilon_{tol}$) it adds the $w_{\mathbf{T}_\mathcal{B}}$ with the highest value to the solution. It then repeats the previous steps, finding a new control input $\mathbf{T}_\mathcal{B}$ that respects the constraints and optimizes for the largest final mass with the extra uncertainty values. If it cannot find any new scenarios that are higher than the given tolerance, it finishes running the algorithm.

6.1.1 Changes to minimisation model

In order to make the minimisation model compatible with the local reduction method, a few changes were needed to be carried out.

To understand the local reduction method, it is essential to understand what is meant by 'scenario'. Each scenario has unique values for uncertainty (in this model $w_{\mathbf{T}_\mathcal{B}}$) and the states ($\mathbf{x}$), but the same control input ($\mathbf{T}_\mathcal{B}$). As the objective for the trajectory optimization is a state itself (-m_f), we instead want to maximise for the worst case scenario mass. This is accomplished by introducing a new variable, γ, and new constraints:

$$\forall k \in \{1, 2, ..., N_{\text{scen}}\}, \quad \gamma \leq m(t_f^k)^k \tag{6.1}$$

where N_{scen} is the number of scenarios, and the objective becomes $\min_{t_f, \mathbf{T}_\mathcal{B}} \quad -\gamma$.

In this way, x now becomes an 'array' of states for every scenario, so that, for example, $\mathbf{r}_\mathcal{I}$ space goes from $\mathbb{R}^{3\times n}$ to $\mathbb{R}^{3\times n\times N_{\text{scen}}}$, and therefore, $\mathbf{x} \in \mathbb{R}^{14\times n\times N_{\text{scen}}}$.

When increasing the number of scenarios, the equality constraints for the final conditions become problematic. This makes intuitive sense: with uncertainty, having all of the scenarios reach the same exact point is most likely physically impossible. To fix this, slack variables are introduced, similar to what is done when transforming from $h(x) = 0$ to $g(x) \leq 0$. For a final condition ζ, the equality constraint $\zeta = \zeta_f$ is transformed into:

$$\forall k \in \{1, 2, ..., N_{\text{scen}}\}. \quad (\zeta^k - \zeta_f)^2 \leq \delta_\zeta \tag{6.2}$$

so that the objective becomes:

$$\min_{t_f, \mathbf{T}_\mathcal{B}} -W_\gamma \gamma + \mathbf{W}_\delta{}^T \delta \tag{6.3}$$

where W_γ and $\mathbf{W}_\delta$ are respectively a positive scalar and vector of weights to be decided by the user.

The overall optimization is shown in Problem 3, where, in discretised form, $\gamma \in \mathbb{R}$, $\delta \in \mathbb{R}^{n_{\text{constr}}}$, $m \in \mathbb{R}^{n\times N_{scen}}$, $\mathbf{r}_\mathcal{I} \in \mathbb{R}^{3\times n\times N_{scen}}$, $\mathbf{v}_\mathcal{I} \in \mathbb{R}^{3\times n\times N_{scen}}$, $q_{\mathcal{B}/\mathcal{I}} \in \mathbb{R}^{4\times n\times N_{scen}}$, $\boldsymbol{\omega}_\mathcal{B} \in \mathbb{R}^{3\times n\times N_{scen}}$ and $\mathbf{T}_\mathcal{B} \in \mathbb{R}^{3\times n}$.

6.1.2 Maximisation models

In terms of the maximisation model, the dimensions are more similar to the original model, except that now the variables to optimize around are the uncertainties and $\mathbf{T}_\mathcal{B}$ **is not a variable**, and is instead a parameter (remains fixed and does not change during the optimization).

where, in discretised form, $m \in \mathbb{R}^n$, $\mathbf{r}_\mathcal{I} \in \mathbb{R}^{3\times n}$, $\mathbf{v}_\mathcal{I} \in \mathbb{R}^{3\times n}$, $q_{\mathcal{B}/\mathcal{I}} \in \mathbb{R}^{4\times n}$, $\boldsymbol{\omega}_\mathcal{B} \in \mathbb{R}^{3\times n}$ and $\mathbf{T}_\mathcal{B} \in \mathbb{R}^{3\times n}$

The maximisation model has no state constraints nor final boundary conditions, to avoid limiting the solution space for $w_{\mathbf{T}_\mathcal{B}}$. There is one maximisation model for every constraint in 3 in addition to the one for the mass maximisation objective, so each G_{max} will be unique. They are summarised in table 6.1.

Problem 3 *Minimization model for the local reduction algorithm, in discretized form*

Cost Function:

$$\underset{\mathbf{T}_{\mathcal{B}}(t)}{\text{minimize}} \quad -W_\gamma \gamma + \mathbf{W}_\delta{}^T \delta$$

s.t, $\forall t \in (0, t_f)$

Dynamics:

$$\begin{aligned}
&\mathbf{T}_{\mathcal{B},w}(t) = w_{\mathbf{T}_{\mathcal{B}}}(t) \cdot \mathbf{T}_{\mathcal{B}}(t) \\
&\dot{m}(t) = -\alpha_{\dot{m}} \|\mathbf{T}_{\mathcal{B},w}(t)\|_2 \\
&\dot{\mathbf{r}}_{\mathcal{I}}(t) = \mathbf{v}_{\mathcal{I}}(t) \\
&\dot{\mathbf{v}}_{\mathcal{I}}(t) = \frac{1}{m(t)} C_{\mathcal{I}/\mathcal{B}}(t) \mathbf{T}_{\mathcal{B},w}(t) + \mathbf{g}_{\mathcal{I}} \\
&\dot{q}_{\mathcal{B}/\mathcal{I}}(t) = \frac{1}{2} \Omega\big(\boldsymbol{\omega}_{\mathcal{B}}(t)\big) q_{\mathcal{B}/\mathcal{I}}(t) \\
&J_{\mathcal{B}} \dot{\boldsymbol{\omega}}_{\mathcal{B}}(t) = [\mathbf{r}_{T,\mathcal{B}} \times]\, \mathbf{T}_{\mathcal{B},w}(t) - [\boldsymbol{\omega}_{\mathcal{B}}(t) \times]\, J_{\mathcal{B}} \boldsymbol{\omega}_{\mathcal{B}}(t)
\end{aligned}$$

Boundary Conditions:

$$\begin{aligned}
m(0) &= m_{wet} \\
\mathbf{r}_{\mathcal{I}}(0) &= \mathbf{r}_{\mathcal{I},i} & (\mathbf{r}_{\mathcal{I}}(t_f) - \mathbf{0})^2 &\leq \delta_r \\
\mathbf{v}_{\mathcal{I}}(0) &= \mathbf{v}_{\mathcal{I},i} & (\mathbf{v}_{\mathcal{I}}(t_f) - \mathbf{v}_{\mathcal{I},f})^2 &\leq \delta_v \\
q_{\mathcal{B}/\mathcal{I}}(0) &= q_{\mathcal{B}/\mathcal{I},i} & (q_{\mathcal{B}/\mathcal{I}}(t_f) - q_{\mathcal{B}/\mathcal{I},f})^2 &\leq \delta_{q\mathcal{B}/\mathcal{I}} \\
\boldsymbol{\omega}_{\mathcal{B}}(0) &= \boldsymbol{\omega}_{\mathcal{B},i} & (\boldsymbol{\omega}_{\mathcal{B}}(t_f) - \mathbf{0})^2 &\leq \delta_{\boldsymbol{\omega}_{\mathcal{B}}} \\
& & \mathbf{e}_2 \cdot \mathbf{T}_{\mathcal{B}}(t_f) &= \mathbf{e}_3 \cdot \mathbf{T}_{\mathcal{B}}(t_f) = 0
\end{aligned}$$

State Constraints:

$$\begin{aligned}
\|\boldsymbol{\omega}_{\mathcal{B}}(t)\|_2 &\leq \omega_{max} \\
\mathbf{e}_1 \cdot \mathbf{r}_{\mathcal{I},f} &\leq \mathbf{e}_1 \cdot \mathbf{r}_{\mathcal{I}}(t) \\
\gamma &\leq m(t_f)
\end{aligned}$$

Control Constraints:

$$\begin{aligned}
T_{min} &\leq \|\mathbf{T}_{\mathcal{B}}(t)\|_2 \leq T_{max} \\
\cos \delta_{max} \|\mathbf{T}_{\mathcal{B}}(t)\|_2 &\leq \mathbf{e}_1 \cdot \mathbf{T}_{\mathcal{B}}(t)
\end{aligned}$$

The first constraint checks whether or not a mass value can be found which is smaller than the value predicted by the minimisation model (equivalent to γ). The rest of the constraints take the same form as the final boundary conditions of the minimisation model, with a different max model for every dimension. Whilst this increases the number of maximisation models, it does not add much time to the algorithm as the models are simple enough that they take 0.1s to run, whilst giving the user the opportunity to tune the constraint violations for each one individually. The constraints are squared so that the value is positive. Lastly, there is one extra constraint for the hard constraint on height. This is effectively more constraining to ensure that the lander does not crash into the surface at any point.

6.1.3 Free-final time vs. fixed-final time

A large difference between the initial formulation and the current formulation is that it is no longer a free final time problem, as can be seen from Problems 3 and 4. This is due to the following: when the problem has free final time, the final time will most likely change depending on the number of scenarios considered. As time-varying uncertainties are parameterised with collocation points, they are intrinsically linked to the final time. If this value varies, the scenario is no longer the worst-case scenario found, and it is likely that for a longer time problem, both the control input,

Problem 4 *General maximisation model*

Cost Function:

$$\underset{w_{\mathbf{T}_\mathcal{B}}}{\text{maximise}} \quad G_{\max}$$

$$\text{s.t, } \forall t \in (0, t_f)$$

Dynamics:

$$\begin{aligned}
\mathbf{T}_{\mathcal{B},w}(t) &= w_{\mathbf{T}_\mathcal{B}}(t) \cdot \mathbf{T}_\mathcal{B}(t) \\
\dot{m}(t) &= -\alpha_{\dot{m}} \|\mathbf{T}_{\mathcal{B},w}(t)\|_2 \\
\dot{\mathbf{r}}_\mathcal{I}(t) &= \mathbf{v}_\mathcal{I}(t) \\
\dot{\mathbf{v}}_\mathcal{I}(t) &= \frac{1}{m(t)} C_{\mathcal{I}/\mathcal{B}}(t) \mathbf{T}_{\mathcal{B},w}(t) + \mathbf{g}_\mathcal{I} \\
\dot{q}_{\mathcal{B}/\mathcal{I}}(t) &= \frac{1}{2} \Omega\big(\boldsymbol{\omega}_\mathcal{B}(t)\big) q_{\mathcal{B}/\mathcal{I}}(t) \\
J_\mathcal{B} \dot{\boldsymbol{\omega}}_\mathcal{B}(t) &= [\mathbf{r}_{T,\mathcal{B}} \times] \, \mathbf{T}_{\mathcal{B},w}(t) - [\boldsymbol{\omega}_\mathcal{B}(t) \times] \, J_\mathcal{B} \boldsymbol{\omega}_\mathcal{B}(t)
\end{aligned}$$

Boundary Conditions:

$$\begin{aligned}
m(0) &= m_{wet} \\
\mathbf{r}_\mathcal{I}(0) &= \mathbf{r}_{\mathcal{I},i} \\
\mathbf{v}_\mathcal{I}(0) &= \mathbf{v}_{\mathcal{I},i} \\
q_{\mathcal{B}/\mathcal{I}}(0) &= q_{\mathcal{B}/\mathcal{I},i} \\
\boldsymbol{\omega}_\mathcal{B}(0) &= \boldsymbol{\omega}_{\mathcal{B},i}
\end{aligned}$$

Max model number	$G_{\max}$
1	$\gamma - m(t_f)$
2-4	$(\mathbf{r}_\mathcal{I}(t_f)_{,i} - \mathbf{r}_{\mathcal{I},f,i})^2 - \delta_{r,i}$
5-7	$(\mathbf{v}_\mathcal{I}(t_f)_{,i} - \mathbf{v}_{\mathcal{I},f,i})^2 - \delta_{v,i}$
8-11	$(q_{\mathcal{B}/\mathcal{I}}(t_f)_{,i} - q_{\mathcal{B}/\mathcal{I},f,i})^2 - \delta_{q,i}$
12-14	$(\boldsymbol{\omega}_\mathcal{B}(t_f)_{,i} - \boldsymbol{\omega}_{\mathcal{B},f,i})^2 - \delta_{\boldsymbol{\omega}_\mathcal{B},i}$
15	$\sum_{i=1}^{n} \mathbf{r}_{\mathcal{I},f,i} - \mathbf{r}_\mathcal{I}(t_f)_{,i}$

Table 6.1: Maximisation model objective values

and the uncertainties that lead to the worst violations, are different. As final time is continuous and in an infinite set, there could be the case where an infinite number of scenarios are needed even for the local reduction method.

In practice, it was found that for a smaller number of maximisation models, the local reduction algorithm described in 1 converged for the free final time problem as well. One reason why this could be the case is due to the similarity in scenarios with similar final times, so that in practice, even if not the same scenario, the constraint becomes active as the final time approaches the value of final time that there was when this situation was found, which would lead to a large increase in the number of scenarios needed to reduce variation to an acceptable level, but still converge with enough scenarios.

However, careful consideration and mathematical theorems need to be developed to prove whether this is true or not. Because of this, and given the scope of this project, it was decided to focus on proving that this method is applicable to a lunar landing scenario with a fixed final time.

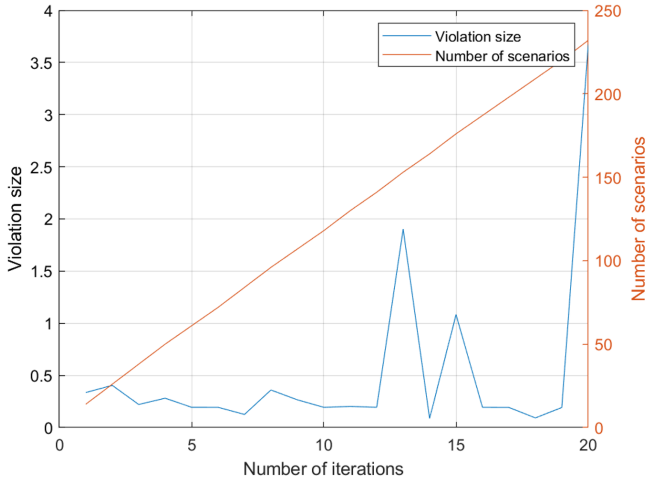

Figure 6.1: Deviation from final conditions with number of scenarios.

6.1.4 JuMP Implementation

Compared to the previous implementation, one extra challenge is the increasing dimensionality of the variables as scenarios are added to the minimisation model. To minimise the overhead experienced from creating and destroying models, the JuMP model was initialised with a number equal to the maximum number of scenarios, and adding the cosntraints progressively. Whilst this means that the initial number of variables is higher, the number of non-zeros in the Hessian remains unaffected. It is therefore overall more efficient than making a new model, adding the constraints and warm-starting it. For the maximisation models, the control input T_B was designed as a parameter, so that it could be changed whilst conserving the objective and constraints.

6.2 Behaviour of algorithm

This results section will focus on the convergence capabilities of the algorithm and prediction for the variability in final values, whilst chapter 7 will compare the open loop and closed loop after propagating the results in a Monte-carlo simulation.

6.2.1 Adding the worst scenario vs all the scenarios above a threshold

Looking at algorithm 1, the reader may have observed that, instead of adding multiple scenarios per iteration as recommended in [15], it instead only adds the scenario with the worst maximum violation at each term. In practice, it was found that this sped up the algorithm, and can be understood by looking at Figure 6.1. In this graph, 'violation size' refers to the objective values of each maximisation model after being normalized, effectively summing all of the deviations from the target final conditions.

As can be observed, the number of scenarios increases extremely fast, with some large peaks due to numerical errors in the minimisation model.

In the original paper, it was found that adding more than one scenario at every iteration reduced the time in running. In practice, for a small problem, the addition of scenarios may only result in a small increase in speed, so that trying to reduce the number of iterations, and therefore the amount of times that the minimisation model has to run, is beneficial. However, for a problem of this size, when every new scenario adds $n \times 15$ new variables, the amount of time per iteration increases quickly against number of scenarios. For the Hessian, this means that $(n \times 15)^2$ scenarios

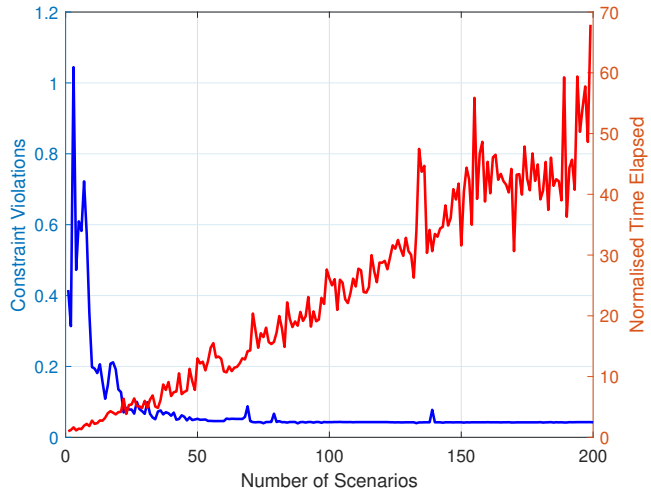

Figure 6.2: Deviation from final conditions with number of scenarios.

are added with each scenario, with the inequality constraints reducing its sparsity. This therefore leads to a large slow down, reducing the benefit of having less iterations, and therefore making it better to optimise the number of scenarios.

Additionally, when adding smaller violations which are not the worst case scenarios, you could be adding constraints that tend to remain inactive and not provide any benefit towards a faster convergence, greatly decreasing the speed at which the algorithm runs. Because of this, only the worst violation was added per iteration.

6.2.2 Number of scenarios vs overall perturbations

For the model described above, with all of the maximisation models, local reduction successfully converged towards smaller perturbations, as shown by Figure 6.2.

As can be observed from the graph, as more scenarios are added, the deviations from the final conditions decrease notably. After about 200 scenarios added, the addition of all of the deviations adds up to 0.042. If it is limited to 50 the relatively small value of 50 scenarios being added for the size of the problem, the perturbations values increase to 0.048. The deviations for each maximisation model can be found in Table 6.2.

As can be seen, the biggest violations can be observed for r2 and the velocities, with no violations expected for the angular velocities and small mistakes in the final position for the quaternion. This means that for an open loop, using this control input, this is the maximum deviations expected. This is a comparatively good result, as it could potentially be run off-board for many scenarios to find a good trajectory, over which could then correct using small corrections.

Section 7 will compare these predictions against the results from the Monte-Carlo simulations.

6.3 Refinements to the model

After having an initial working implementation, some further refinements were thought of for the model.

6.3.1 Tuning parameters

For the results seen in Subsection 6.2.2, all maximisation models were given the same weight, which can be quite a naive way of doing things. To give an example, for an error of $[0, 0, \pm 0.05, \pm 0.05]$,

Table 6.2: Predicted deviations from local reduction algorithm after the addition of 50 scenarios.

Name	Value	+/- Value	
m	0	0	kg
$\mathbf{r}_{\mathcal{I},1}$	0	0	m
$\mathbf{r}_{\mathcal{I},2}$	0.01055	10.27 m	
$\mathbf{r}_{\mathcal{I},3}$	0.004803	6.930	m
$\mathbf{v}_{\mathcal{I},1}$	0.01035	1.017	m/s
$\mathbf{v}_{\mathcal{I}}, 2$	0.004364	0.6606	m/s
$\mathbf{v}_{\mathcal{I},3}$	0.005485	0.7406	m/s
$q_{\mathcal{B}/\mathcal{I},1}$	0	0	
$q_{\mathcal{B}/\mathcal{I},2}$	0	0	
$q_{\mathcal{B}/\mathcal{I},3}$	0.002593	0.05092	
$q_{\mathcal{B}/\mathcal{I},4}$	0.003061	0.05532	
$\boldsymbol{\omega}_{\mathcal{B},1}$	0	0	rad/s
$\boldsymbol{\omega}_{\mathcal{B},2}$	0	0	rad/s
$\boldsymbol{\omega}_{\mathcal{B},3}$	0	0	rad/s
Overall	**0.04238**	**0.5201**	

the perturbation is shown in Figure 6.3. This is clearly a very small error and less important than the errors in velocity, which could be disregarded for a landing problem. Additionally, the optimisation is currently constraining the rotation of the spacecraft on touchdown, which is also something that, for the vast majority of missions, may be over-constraining.

It was therefore found that, for better tuning of the constraints to include, the local reduction method could be made more performant, and achieve better results with less scenarios. Some qualitative analysis is shown in table 6.3, as well as some values, that could be further optimised by an experienced user.

The key relaxations are as follows: the constraint on the final vertical velocity is changed so that the final conditions is instead the final value - for a lunar landing, the key is to make sure it doesn't land too hard, but as the mission finishes above and then hovers down after engine switch off, this is not a large problem. Regarding the slack variable constraints, more importance are given to the velocities than to the positional ones, as the former can threaten the mission whilst the latter, assuming there are no obstacles in the vicinity, are desirable but not essential. Finally, the quaternion constraints are changed and reduced, so that it is free to rotate in the x (vertical) direction. This can be achieved by allowing the x imaginary value ($q_{\mathcal{B}/\mathcal{I},1}$) and the scalar value ($q_{\mathcal{B}/\mathcal{I},4}$) to rotate around each other.

This is just an example with some ideas considered. One advantage of this local reduction implementation is that future users will have freedom to choose the parameters considering what is important for their missions and objectives.

6.3.2 Limiting number of scenarios

For the closed loop implementation, with more information in Chapter 7, it was thought that the number of iterations could be further reduced. As the local reduction method will be rerun at each time-step, the final landing position will decrease naturally, so that the prediction for the open-loop response in the first time-step is less important.

Looking at Figure 6.2, it can be observed that the gradient of violations vs. iterations becomes shallower around the 25 iteration mark, so this value was selected for the closed loop formulation. This increases the prediction for constraint violations to around 0.08, and decreases the running time to complete the local reduction run from 1:30 hours to 25 minutes. As will be observed, this

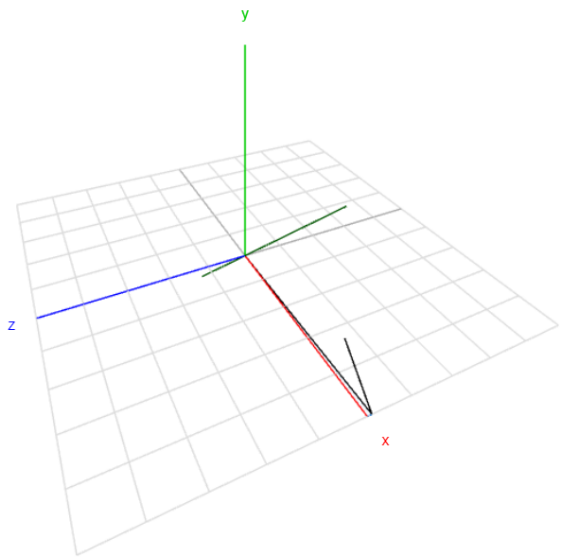

Figure 6.3: Perturbation in quaternion, where the objective is to be colinear with the x-axis direction.

Table 6.3: Relaxed constraints for optimisation problem

Element	Constraint	Weighting
$\mathbf{r}_{\mathcal{I},1}$	Hard constraint - larger than minimum required AND soft constraint to minimise deviation.	5
$\mathbf{r}_{\mathcal{I},2}$	Soft - minimise deviation, not essential.	2
$\mathbf{r}_{\mathcal{I},3}$	Soft - minimise deviation, not as important.	2
$\mathbf{v}_{\mathcal{I},1}$	Smaller than target value (very soft)	2
$\mathbf{v}_{\mathcal{I}}, 2$	Soft - minimise deviation.	3
$\mathbf{v}_{\mathcal{I},3}$	Soft - minimise deviation.	3
$q_{\mathcal{B}/\mathcal{I}}, 1$ & $q_{\mathcal{B}/\mathcal{I}}, 4$	Slack variable - addition to 1	4
$q_{\mathcal{B}/\mathcal{I}}, 2$	Slack variable (strong)	4
$q_{\mathcal{B}/\mathcal{I}}, 3$	Slack variable (strong)	4
$\boldsymbol{\omega}_{\mathcal{B},1}$	Slack variable	3
$\boldsymbol{\omega}_{\mathcal{B},2}$	Slack variable	3
$\boldsymbol{\omega}_{\mathcal{B},3}$	Slack variable	3

does not greatly affect the final results.

7 Results & Discussion

To validate the local reduction method and compare it against the baseline (the model in Section 4), Monte Carlo simulations were run for the open-loop and closed-loop cases.

7.1 Open-Loop Response - Key Findings

A Monte Carlo simulation was set up to see how the original control input (with one scenario) compares to the control input with 50 scenarios. The pseudo-code for this is shown below.

Listing 7.1: Monte-carlo setup

```
j 1:n_montecarlo
  T_d_random = generateTd(n, dL, dU)
  finalvals_OL, allvals_OL = propagate!(T_d_random, T_B)
```

It was run for a total of 2000 iterations. The results can be observed in Table 7.1.

Table 7.1: Results from 2000 Monte-carlo runs, comparing the final conditions achieved.

Variable	Mean		Standard deviation		Max deviation	
	Original	LR	Original	LR	Original	LR
$\mathbf{r}_{\mathcal{I},1}$ [m]	2.192	14.52	18.81	2.910	63.84	23.70
$\mathbf{r}_{\mathcal{I},2}$ [m]	-1.802	0.9728	31.96	3.714	130.1	10.43
$\mathbf{r}_{\mathcal{I},3}$ [m]	1.813	-0.8870	26.60	1.141	98.74	4.468
$\mathbf{v}_{\mathcal{I},1}$ [m/s]	-3.421	-3.687	2.301	0.1898	9.468	0.6015
$\mathbf{v}_{\mathcal{I},2}$ [m/s]	-0.2720	2.127e-02	3.677	0.1504	14.08	0.4664
$\mathbf{v}_{\mathcal{I},3}$ [m/s]	3.788e-02	-1.411e-02	3.208	0.1131	12.01	0.4880
$q_{\mathcal{B}/\mathcal{I},1}$	5.618e-03	5.353e-03	7.373e-03	9.285e-03	2.819e-02	3.355e-02
$q_{\mathcal{B}/\mathcal{I},2}$	4.360e-04	1.263e-03	4.343e-03	2.643e-03	1.420e-02	9.880e-03
$q_{\mathcal{B}/\mathcal{I},3}$	-2.580e-03	-2.400e-03	4.532e-03	4.650e-03	1.642e-02	1.632e-02
$q_{\mathcal{B}/\mathcal{I},4}$	1.001	0.9904	1.750e-04	6.080e-04	1.773e-03	1.188e-02
$\boldsymbol{\omega}_{\mathcal{B},1}$ [rad/s]	-8.950e-11	3.420e-24	3.960e-09	2.180e-22	1.770e-07	9.670e-21
$\boldsymbol{\omega}_{\mathcal{B},2}$ [rad/s]	-4.100e-04	-9.900e-04	4.962e-03	2.147e-03	1.683e-02	8.341e-03
$\boldsymbol{\omega}_{\mathcal{B},3}$ [rad/s]	-2.890e-03	-2.910e-03	5.202e-03	7.996e-03	1.820e-02	2.922e-02
m [kg]	1864	1829	72.28	6.010e-02	175.2	0

The results in Table 7.1 prove the impact that local reduction can have even for the open-loop case, and therefore its applicability for a landing problem. As can be observed, the standard deviation for the local reduction method is at least one order of magnitude smaller for all of the values that experience large deviations in the original model. The best result can be observed for the mass values, reducing the standard deviation in the mass from 72.28kg to 0.601kg.

This point is further amplified by the mass's maximum deviations from the target value, which shows no final fuel masses below the expected value. If this result is verified in future papers, it

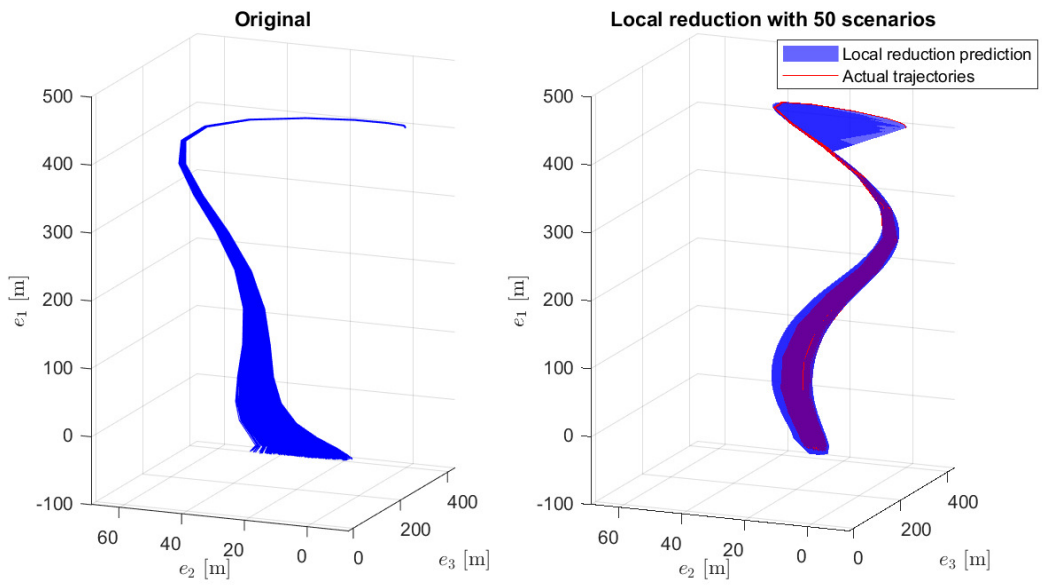

Figure 7.1: Comparison between the nominal control input and the local reduction input, for a 2000 simulation Monte Carlo.

could reduce the fraction of fuel on-board due to uncertainty massively. For the final position and velocities, the maximum deviation are at least an order of magnitude lower for the final positions and velocities, which are the variables where the original open loop gives the most extreme results, whilst also improving on already small deviations such as for $\boldsymbol{\omega}_{\mathcal{B},2}$. Figure 7.1 shows the spread of trajectories in the original control input compared to the local reduction input. Additionally, a coarse surface mesh was created based on the worst case trajectories found by the local reduction algorithm. Whilst only optimising for the final conditions, it can be observed that the worst case trajectories bound most trajectories that are actually observed extremely well, with only some trajectories being outside momentarily, potentially due to the coarseness in the mesh created. This shows one of the best results observed in the local reduction method: its ability to accurately predict the worst case scenarios, further described in the next section.

7.1.1 Accuracy in predictions

In addition to improving the performance, it is also of note the accuracy of the predictions of violations for the maximisation models, which can be observed in Table 7.2.

As can be seen, the predictions are generally quite accurate for most values. In some cases, the predictions are slightly larger than the actual value found by the Monte Carlo, arguably because the Monte Carlo could need more runs to find the worst case scenario.

The only notable exception is for $\mathbf{r}_{\mathcal{I},1}$, as there was no max model to reduce the maximum deviation observed in this version, so the algorithm favoured violations on the upper bound to avoid the hard constraint on altitude.

Considering that the maximisation step in the local reduction algorithm takes $0.5s$ to run, this finding by itself could have great impact, as for many situations where a Monte Carlo cannot be run or is a significant expense, an easy to setup maximisation model could give very close results. Arguably, if the model is well defined and random restarts are used, it is likely that a better estimate for the worst possible case can be found than when compared to relatively small Monte Carlo runs, as there is always the chance that it has not been run for long enough and therefore

Local reduction may therefore a method that can not only reduce the number of scenarios that

Table 7.2: Predicted values by the local reduction in Chapter 6 with the real values from the Monte Carlo simulation.

Variable		Predicted value	Actual value
$\mathbf{r}_{\mathcal{I},1}$	m	0.4395	**23.70**
$\mathbf{r}_{\mathcal{I},2}$	m	10.28	10.43
$\mathbf{r}_{\mathcal{I},3}$	m	6.930	4.468e
$\mathbf{v}_{\mathcal{I},1}$	ms^{-1}	1.017	0.6015
$\mathbf{v}_{\mathcal{I}}, 2$	ms^{-1}	0.6607	0.4664
$\mathbf{v}_{\mathcal{I},3}$	ms^{-1}	0.7406	0.4880
$q_{\mathcal{B}/\mathcal{I},1}$		9.960e-05	3.355e-02
$q_{\mathcal{B}/\mathcal{I},2}$		1.235e-02	9.880e-03
$q_{\mathcal{B}/\mathcal{I},3}$		5.092e-02	1.632e-02
$q_{\mathcal{B}/\mathcal{I},4}$		5.532e-02	1.188e-02
$\boldsymbol{\omega}_{\mathcal{B},1}$	rads^{-1}	1.430e-02	9.670e-21
$\boldsymbol{\omega}_{\mathcal{B},2}$	rads^{-1}	2.613e-02	8.341e-03
$\boldsymbol{\omega}_{\mathcal{B},3}$	rads^{-1}	1.399e-02	2.922e-02
m	kg	0	0

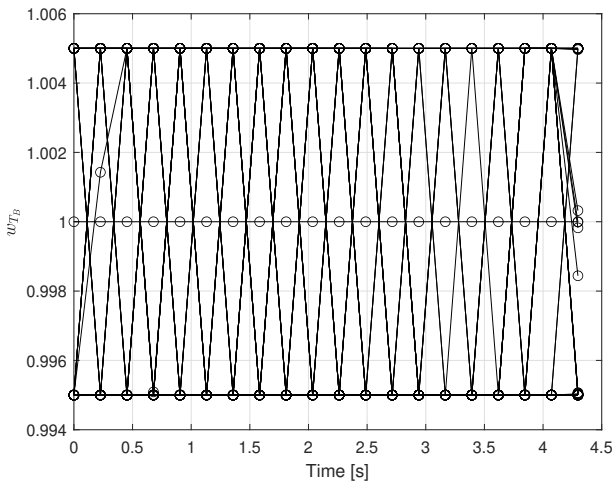

Figure 7.2: $w_{\mathbf{T}_{\mathcal{B}}}$ for all of the scenarios considered

must be considered, but also provide an accurate measure of the deviations that can be expected. This is an avenue that must be investigated further and proven, and for which a range of use cases are suggested in Chapter 8.

7.1.2 Scenarios considered

In literature [15], it was shown that the local reduction method can find scenarios that are not in the bounds, showing the non-robustness of the extreme scenarios method for these cases and other scenario-based methods.

To illustrate whether this also happens for this model, the values for $w_{\mathbf{T}_{\mathcal{B}}}$ are plotted in Figure 7.2. As can be observed, the local reduction method also finds results that are not in the bounds. This means that it is likely that any scenario-based method that takes the values at the bounds will miss some of the worst case scenarios and therefore have worse performance than the local reduction method.

In addition to providing better results, it can also be a much smaller problem than the extreme

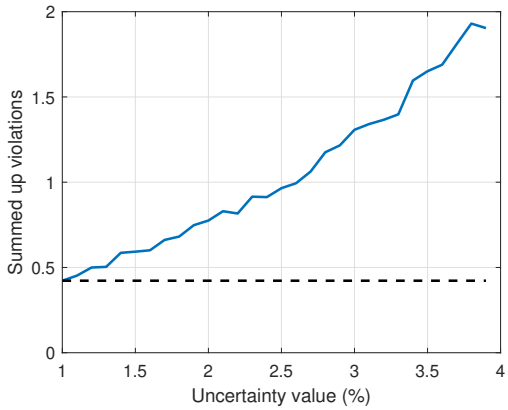

Figure 7.3: Sensivity analysis on the Open-Loop Local Reduction control input

scenario situation. For time varying uncertainty, if all of the combinations between the lower and upper bound for uncertainty want to be considered, then $N_{scen} = 2^n$. For 20 collocation points as considered in this problem, this would equal 1'048'576 scenarios! For the way the optimisation has been implemented, it is unlikely that the computations required for a Hessian of such size could even be computed (as a back-of-the-envelope calculation, can expect a Hessian of size $(2^n \times n \times 15)^2 = 9.9e16$.)

Therefore, local reduction can be expected to give better results with a much smaller number of scenarios.

7.1.3 Sensitivity analysis

As mentioned in Section 5, it is likely that for some uncertainties, knowing where to place the bound is hard. Because of this, it is even more important than usual how sensitive the problem is to larger-than-expected disturbances. The Monte Carlo simulation was repeated for a range of values for $w_{\mathbf{T}_\mathcal{B}}$ between the original (1%, equivalent to $\pm 0.5\%$), and 5%. The overall, normalised violations are shown in Figure 7.3. As can be observed, the deviations increase linearly with a gradient less than one, showing the system is stable. This is an intuitive finding; it is expected that the control input that minimises the effect of oscillations in thrust will also provide good results when this disturbance grows in size.

7.2 Closed-Loop Response

For the closed-loop model, a decreasing horizon strategy was used. It had a similar setup as used for the open-loop model, except that now the generated values for disturbances are progressively added to the optimisation model, as the control input for those time-steps are fixed. This in turn "freezes" the states to the values that have already been found, allowing the model to change the remaining free variables of the control input. This is shown in Algorithm 12.

where dL and dU are the lower and upper bounds of the uncertainty respectively.

Similarly to the open-loop values, parameters can be calculated and observed in Table 7.3. For the closed-loop response, improved results can be observed for both systems. In the case of the nominal case, standard deviations more than halve, whilst in the case of the local reduction method, they are minimised below the desired target, getting to cm level accuracies for altitudes and mm/s level for the velocities. In terms of deviations, the maximum distance from the target observed for the local reduction is of 10-20cm in each axis, whilst the velocities have a maximum deviation 1-2cm/s in each axis. These are very good results below the target deviations set for

Algorithm 2: One iteration of the closed-loop model for the Monte Carlo simulation.

Output: $\mathbf{x}$, $\mathbf{T}_{\mathcal{B}}$ for random uncertainties
Set $w_{\mathbf{T}_{\mathcal{B}}} \leftarrow 1_n$
Solve Problem 2 / Run Algorithm 1.
forall $i \in \{1, 2\}$ **do**
  Fix $\mathbf{T}_{\mathcal{B},i}$
  Set $w_{\mathbf{T}_{\mathcal{B}},i} \sim \text{rand}(dL, dU)$
end
forall $i \in \{3, 4, ..., n\}$ **do**
  Solve Problem 2 / Run Algorithm 1.
  Fix $\mathbf{T}_{\mathcal{B},i}$
  Set $w_{\mathbf{T}_{\mathcal{B}},i} \sim \text{rand}(dL, dU)$
end
Solve Problem 2 / Run Algorithm 1.

Table 7.3: Results from 2000 Monte Carlo runs, comparing the final conditions achieved for the closed loop

Variable	Mean		Standard deviation		Max deviation	
	Original	LR	Original	LR	Original	LR
$\mathbf{r}_{\mathcal{I},1}$ [m]	7.133e-01	1.542e-02	9.009	1.645e-02	24.88	1.071e-01
$\mathbf{r}_{\mathcal{I},2}$ [m]	-3.736e-01	-9.353e-04	9.866	2.281e-02	32.01	1.980e-01
$\mathbf{r}_{\mathcal{I},3}$ [m]	-1.383e-01	-6.476e-05	7.063	3.256e-02	13.26	1.257e-01
$\mathbf{v}_{\mathcal{I},1}$ [m/s]	-3.296	-3.679	1.552	1.576e-03	3.044	2.056e-02
$\mathbf{v}_{\mathcal{I}}, 2$ [m/s]	5.545e-02	-1.033e-04	1.001	4.663e-03	2.376	1.039e-02
$\mathbf{v}_{\mathcal{I},3}$ [m/s]	-3.428e-03	-4.112e-04	9.326e-01	2.519e-03	1.615	1.451e-02
$q_{\mathcal{B}/\mathcal{I},1}$	3.014e-03	-1.065e-04	3.395e-02	2.262e-04	0.0661	1.172e-03
$q_{\mathcal{B}/\mathcal{I},2}$	-2.809e-03	-3.039e-04	3.701e-02	3.340e-04	0.0679	1.694e-03
$q_{\mathcal{B}/\mathcal{I},3}$	1.947e-03	-2.883e-04	2.622e-02	3.650e-04	5.295e-02	1.878e-03
$q_{\mathcal{B}/\mathcal{I},4}$	0.999	1.001	1.593e-02	1.189e-03	4.422e-02	1.429e-02
$\boldsymbol{\omega}_{\mathcal{B},1}$ [rad/s]	-4.83e-10	1.42e-12	5.11e-09	1.28e-10	8.62e-08	5.41e-09
$\boldsymbol{\omega}_{\mathcal{B},2}$ [rad/s]	4.28e-03	2.09e-06	9.58e-02	2.63e-04	2.27e-02	1.45e-03
$\boldsymbol{\omega}_{\mathcal{B},3}$ [rad/s]	-8.83e-03	-6.82e-06	8.52e-02	8.79e-04	2.04e-02	2.27e-03
m [kg]	1886	1876	9.875	6.534	N/A	N/A

the local reduction method, and shows that, assuming that local reduction could be applied in real-time, extremely fine pin-point landing accuracies could be met.

The max deviation values for the nominal case show improvement but not as much as could be expected. This could potentially be due to numerical errors in the decreasing horizon implementation, as shown in Figure 7.4. In some cases, there was a numerical error when recomputing the control input, causing it to apply a control input that was not optimal, leading to very large deviations from the target point. Whilst most of the obvious numerical errors were removed from the values in the table, it is likely that some remain, which could explain the larger value than expected, and therefore cannot be directly compared to the local reduction method. Nevertheless, the local reduction method exhibits extremely accurate pin-point landing for the closed loop case as shown in the graph.

7.2.1 Feedback vs computational time

Whilst the effectiveness of the local reduction method has been proven for the open-loop policy, and in terms of validation mechanisms, the results for the closed-loop should be taken with a pinch

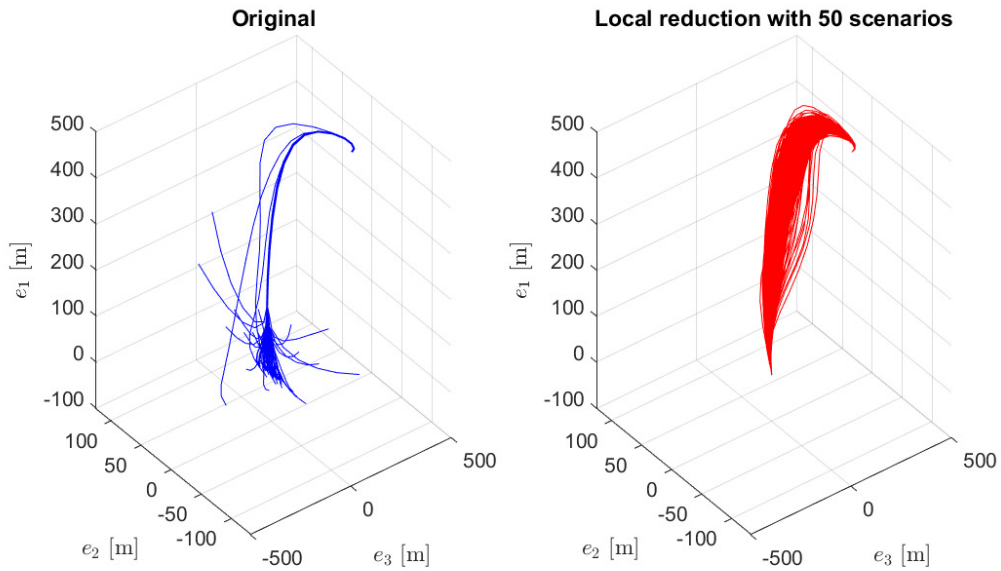

Figure 7.4: Comparison between the nominal control input and the local reduction input, for a 2000 simulation Monte Carlo in closed-loop.

of salt.

In the way closed-loop has been defined, the local reduction method must be run $(n-1)$ number of times. This means that for the case shown, with $n = 20$, it takes about 25 minutes to run, and is therefore an unfair comparison against the original method, which could run in real time.

This approach to closed-loop local reduction was considered the first (somewhat naive) approach, and others were thought of that could be developed further.

One approach considered was to use the local reduction method to train a lookup table. One approach would be to unlink T_B for each scenario and to have a unique one for each uncertainty. However, in order to have a unique T_B for every possible uncertainty with feedback, in its discrete form $T_B \in \mathbb{R}^{n \times N^n_{\text{scen}}}$, which for 50 scenarios and 20 collocation points equals $9.5367x10^{33}$. Another potential method that could store this information more efficiently would be to substitute the discrete array of T_B values for a neural network that took as input the previous states and uncertainty values to give a control input. As the minimisation optimisation would train the weights directly and the maximisation step would find cases for which the output does not give good results, it is possible that the neural network could be trained without having to generate data separately, using overall a smaller number of scenarios than required to train a neural network in the traditional fashion, and potentially, with a much more accurate and robust idea of the potential deviations from the target values. It could therefore allow for a real-time feedback policy that is robust.

Finally, in a similar sentiment, it is possible to redefine the control input $\mathbf{T}_{\mathcal{B}}$ so that it also includes a number of weights based on previous values, of which the most basic form may be considered to be:

$$u = \mathbf{T}_{\mathcal{B}} + \mathbf{K} \cdot \mathbf{x} \tag{7.1}$$

where $\mathbf{K} \in \mathbb{R}^{n \times n \times 15}$ is a matrix of weights to multiply by the past states.

Due to the time-constraint of this project, none of these methods were developed far enough to get tangible results, and further work must be carried out in this direction to truly show the potential of this methodology when it exploits feedback policy effectively.

7.2.2 Alternative formulations

Whilst the formulation used proves the usefulness, applicability and potential impact that this method can have in lunar landing, and trajectory optimisation more generally, it may be that there are other combinations that could showcase this more effectively.

One possibility that was considered and would want to explore further was using a convex formulation in conjunction with local reduction. Whilst it has been proven that local reduction works for the most general case which is good, it is likely that, with a convex model that can converge in a small number of iterations, local reduction could greatly improve its running speed, potentially leading the way towards a lr-MPC. Additionally, in case there is any inaccuracies or errors introduced by the convexification of the problem, the error could be included as an extra uncertainty, simultaneously increasing the speed of the local reduction, and the robustness of the convex model.

7.2.3 Propagation

To propagate the trajectories for the closed-loop model, other propagators were developed, such as using Runge-Kutta to integrate the ODEs with small error. When using this method, larger deviations were observed even for the case with no disturbance. In the end, it was propagated within JuMP to isolate the effect of the uncertainties.

However, the propagator showed that the discretisation error is not negligible, and therefore in the real-world the values for error will be significantly higher. Because of this, further work must be carried out to validate these results outside of theory.

8
Future Work & Use Cases

This project has proven the potential of the local reduction method for soft landing, opening many avenues of research. This section details the immediate next steps as well as its many potential use cases.

As described in Section 7, the immediate next step should be to finish the development of the improved closed-loop versions of the local reduction method. This should be done by expanding the control input to include data from the previous steps, and tested by modelling different closed-loop models. By including feedback, for example, local reduction could change the timing of large burns to minimise uncertainty overall, which has the potential of achieving better final conditions than observed with the naive closed loop approach whilst doing all of the computation offline.

Another area of research for further work on is the free final time problem. In the practical sense, the numerical issues that arise in the current implementation should be investigated. It must be proven whether or not a free final time problem remains robust theoretically. Whilst the objective of the project was to prove its applicability, some work was also done towards improving the speed of the implementation. Further efforts should be dedicated to this, especially on warm-starting and initial guesses, which, as described in Section 2, has great potential to speed NLP algorithms.

Finally, now that the local reduction method for a NLP has been shown to work, combinations with other technologies should be explored to exploit all the potential advantages of this technology. A next step that would have been taken if not for the time constraint would be to see how local reduction faires with convex models, and testing the hypothesis that it would be a win-win scenario; simultaneously increasing the running speed whilst including the error introduced by the convexification as an uncertainty.

Afterwards, more potential combinations should be explored. A key conclusion from this project was the potential to predict Monte Carlo simulations in a fraction of a second. More work should be carried out towards proving this claim and exploring how this could be used. Monte Carlo simulations are used in a wide range of industries, and having an alternative that can run in 0.5 seconds could change the paradigm of validation.

To illustrate, with real-time validation, the maximisation step could be run in real-time for planned trajectories to ensure that there are no violation of hard constraints, for example for obstacle avoidance. Local reduction could also be used to train neural networks very quickly. By substituting the thrust variable for a neural network, the control input response would become convex, potentially decreasing the running time of the local reduction. Maximisation models would then add scenarios to train the neural network, training it on the most useful next scenario, and therefore training the neural network more quickly and avoiding over-fitting. Finally, local reduction would provide guarantees on the neural network's performance and robustness, mitigating one of the largest problems neural networks face currently for adoption.

One of the most interesting future applications is in its combination with probabilistic deep learning. The development of probabilistic deep learning (PDL) is allowing ML models to output distributions, giving a measure of uncertainty and confidence. One prominent approach is Bayesian Neural Networks (BNNs), which explicitly model the posterior distribution over the model parameters [24]. By using BNNs as surrogate models for hard-to-model dynamics [25], and using the

uncertainty values outputted as input for the local reduction method, an accurate, feasible trajectory that adheres to worst-case scenarios could be found. This strategy can be further enhanced by integrating convex methods, resulting in a reliable, adaptable, and robust MPC algorithm. Local reduction, therefore, not only exhibits significant promise and versatility in a range of applications, but could potentially serve as a crucial component in achieving the ultimate goal of robust, adaptable, real-time trajectory planning in unknown environments.

Conclusions

This project aimed to discover whether the local reduction method could be used to find robust trajectories for powered descent in lunar landing, as well as investigating how its accuracy and speed compared against other robust methods, and whether it can deal with all the types of uncertainties in lunar landing.

To do this, two nonlinear models were developed to characterise lunar landing, the main sources of uncertainties in lunar landing found and quantified, a local reduction algorithm developed, and Monte Carlo approaches devised to benchmark both the open-loop and closed-loop responses.

It was found that local reduction method, in open-loop, reduced the maximum deviations caused by the uncertainties by at least one order of magnitude for all constraints considered, decreasing the variance observed in the final responses by one order of magnitude, and notably reducing the standard deviation in the mass from 72.28kg to 0.601kg. For the closed-loop response, it was shown that a real-time implementation of the local reduciton would achieve a lunar landing ellipse in the centimeter scale. Most interestingly, it was found that the maximisation step could quite accurately predict the results observed in the Monte Carlo simulation.

Therefore, this project proves that local reduction is applicable for lunar landing scenarios, and that generally increased the accuracy when compared to the nominal optimal solution for a NLP, which can be considered the state-of-the-art. In terms of speed, whilst due to the time constraints of the project, it was not directly compared against other methodologies, it was shown that, for this problem, common scenario approaches would miss worst-case scenarios, and shown that local reduction provides better solutions with less scenarios. Whilst not implemented in the current version, the project also describes how space-varying uncertainties may be included, therefore showing in principle that local reduction could deal with all the types of uncertainties present in lunar landing.

This project has showcased the potential impact that local reduction can have not just in lunar landing but in trajectory optimisation more generally, and the range of use cases, from validation, faster training and learning, or potentially becoming one of the building blocks in the pursuit of adaptable, fast and robust guidance and motion planning.

However, some more work is needed to corroborate the findings here found. The current closed-loop implementation could not be run in real-time, therefore not being comparable to the nominal case, and therefore further work on the closed-loop formulation is required. It has only been tested in the same way the model is built, and therefore the theoretical results may not translate to the real world. Extra work is needed to have a free final-time implementation, as well as further work on the speed of the algorithm. Future research should therefore focus on covering these bases, as well as on combinations with other methodologies to fully exploit the potential of local reduction in optimal control.

Bibliography

[1] *List of missions to the moon*, Mar. 2024. [Online]. Available: https://en.wikipedia.org/wiki/List_of_missions_to_the_Moon.

[2] S. Biswas and S. Biswas, "Chandrayaan 3 and other moon missions: An update and review," *SSRN Electronic Journal*, 2023. DOI: 10.2139/ssrn.4553155.

[3] L. Blackmore, "Autonomous precision landing of space rockets - lars blackmore," in (Frontiers of Engineering), Frontiers of Engineering. National Academies Press, Jan. 2017, pp. 33–42, ISBN: 9780309450362. [Online]. Available: https://www.nap.edu/read/23659/chapter/10.

[4] "Japan's high-precision 'moon sniper' spacecraft hit its target, but it landed upside down," *LiveScience*, Jan. 2024.

[5] Z.-y. Song, C. Wang, S. Theil, *et al.*, "Survey of autonomous guidance methods for powered planetary landing," *Frontiers of information technology electronic engineering*, vol. 21, no. 5, pp. 652–674, May 2020. DOI: 10.1631/FITEE.1900458. [Online]. Available: https://link.springer.com/article/10.1631/FITEE.1900458.

[6] A. R. Klumpp, "Apollo lunar descent guidance," *Automatica (Oxford)*, vol. 10, no. 2, pp. 133–146, Jan. 1974. DOI: 10.1016/0005-1098(74)90019-3. [Online]. Available: https://dx.doi.org/10.1016/0005-1098(74)90019-3.

[7] R. Prakash, P. D. Burkhart, A. Chen, *et al.*, "Mars science laboratory entry, descent, and landing system overview," IEEE, 2008, pp. 1–18, ISBN: 1095-323X. DOI: 10.1109/AERO.2008.4526283. [Online]. Available: https://ieeexplore.ieee.org/document/4526283.

[8] R. Sostaric and J. Rea, "Powered descent guidance methods for the moon and mars," in (AIAA Guidance, Navigation, and Control Conference and Exhibit), AIAA Guidance, Navigation, and Control Conference and Exhibit. American Institute of Aeronautics and Astronautics, Jun. 2005. DOI: 10.2514/6.2005-6287.

[9] F. Fahroo and I. M. Ross, "Pseudospectral methods for infinite-horizon nonlinear optimal control problems," *Journal of guidance, control, and dynamics*, vol. 31, no. 4, pp. 927–936, Jul. 2008. DOI: 10.2514/1.33117. [Online]. Available: http://arc.aiaa.org/doi/full/10.2514/1.33117.

[10] B. Açıkmeşe and L. Blackmore, "Lossless convexification of a class of optimal control problems with non-convex control constraints," *Automatica (Oxford)*, vol. 47, no. 2, pp. 341–347, Feb. 2011. DOI: 10.1016/j.automatica.2010.10.037. [Online]. Available: https://dx.doi.org/10.1016/j.automatica.2010.10.037.

[11] Y. Guo, M. Hawkins, and B. Wie, "Applications of generalized zero-effort-miss/zero-effort-velocity feedback guidance algorithm," *Journal of Guidance, Control, and Dynamics*, vol. 36, no. 3, pp. 810–820, May 2013. DOI: 10.2514/1.58099. [Online]. Available: http://arc.aiaa.org/doi/full/10.2514/1.58099.

[12] J. B. Rawlings, D. Q. Mayne, and M. Diehl, *Model predictive control*, 2nd edition. Madison, Wisconsin: Nob Hill Publishing, Oct. 2017, ISBN: 0975937731. [Online]. Available: http://bvbr.bib-bvb.de:8991/F?func=service&doc_library=BVB01&local_base=BVB01&doc_number=030095384&sequence=000001&line_number=0001&func_code=DB_RECORDS&service_type=MEDIA.

[13] G. C. Calafiore and M. C. Campi, "The scenario approach to robust control design," *IEEE Transactions on Automatic Control*, vol. 51, no. 5, pp. 742–753, May 2006. DOI: 10.1109/TAC.2006.875041. [Online]. Available: https://ieeexplore.ieee.org/document/1632303.

[14] D. Q. Mayne, E. C. Kerrigan, E. J. van Wyk, and P. Falugi, "Tube-based robust nonlinear model predictive control," *International journal of robust and nonlinear control*, vol. 21, no. 11, pp. 1341–1353, Jul. 2011. DOI: 10.1002/rnc.1758. [Online]. Available: https://api.istex.fr/ark:/67375/WNG-MC43TPGP-D/fulltext.pdf.

[15] M. Zagorowska, P. Falugi, E. O'Dwyer, and E. C. Kerrigan, "Automatic scenario generation for efficient solution of robust optimal control problems," *International journal of robust and nonlinear control*, vol. 34, no. 2, pp. 1370–1396, Oct. 2023. DOI: 10.1002/rnc.7038. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/rnc.7038.

[16] D. Malyuta, Y. Yu, P. Elango, and B. Açıkmeşe, "Advances in trajectory optimization for space vehicle control," *Annual reviews in control*, vol. 52, pp. 282–315, 2021. DOI: 10.1016/j.arcontrol.2021.04.013. [Online]. Available: https://dx.doi.org/10.1016/j.arcontrol.2021.04.013.

[17] M. Kelly, "An introduction to trajectory optimization: How to do your own direct collocation," *SIAM review*, vol. 59, no. 4, pp. 849–904, Dec. 2017. DOI: 10.1137/16M1062569. [Online]. Available: https://www.jstor.org/stable/45109257.

[18] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017. DOI: 10.1137/141000671. [Online]. Available: https://epubs.siam.org/doi/10.1137/141000671.

[19] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization," *Mathematical Programming Computation*, 2023. DOI: 10.1007/s12532-023-00239-3.

[20] F. Capolupo and A. Rinalducci, "Descent landing trajectory and guidance algorithms with divert capabilities for moon landing," Cornell University Library, arXiv.org, Tech. Rep., Dec. 2023. DOI: 10.48550/arxiv.2305.13846. [Online]. Available: https://search.proquest.com/docview/2818550031.

[21] M. Szmuk and B. Acikmese, "Successive convexification for 6-dof mars rocket powered landing with free-final-time," in (2018 AIAA Guidance, Navigation, and Control Conference), 2018 AIAA Guidance, Navigation, and Control Conference. Reston: American Institute of Aeronautics and Astronautics, Jan. 2018. DOI: 10.2514/6.2018-0617. [Online]. Available: https://search.proquest.com/docview/2560873895.

[22] E. Anzaline, E. Braden, N. Ahmad, J. Everett, and K. Miller, "Guidance and navigation design trades for the lunar pallet lander," Marshall Space Flight Center, Feb 1, 2019. [Online]. Available: https://ntrs.nasa.gov/citations/20190002112.

[23] H. Wang and H. Liu, "Stochastic simulation method of performance reliability estimation on liquid propellant rocket engine," *Huojian Tuijin / Journal of Rocket Propulsion*, vol. 32, no. 4, pp. 26–30, Aug. 2006. [Online]. Available: https://search.proquest.com/docview/30024387.

[24] L. V. Jospin, H. Laga, F. Boussaid, W. Buntine, and M. Bennamoun, *Hands-on bayesian neural networks-a tutorial for deep learning users*, May 2022. DOI: 10.1109/MCI.2022.3155327. [Online]. Available: https://ieeexplore.ieee.org/document/9756596.

[25] G. Ryou, E. Tal, and S. Karaman, "Multi-fidelity black-box optimization for time-optimal quadrotor maneuvers," *The International journal of robotics research*, vol. 40, no. 12-14, pp. 1352–1369, Dec. 2021. DOI: 10.1177/02783649211033317. [Online]. Available: https://journals.sagepub.com/doi/full/10.1177/02783649211033317.